TECHNICAL WHITEPAPER · REVIEWED EDITION

Debt Reality Checker

Architectural Whitepaper, Algorithmic Solvency Engine & Technical Implementation Specification

with an independent technical review, algorithm v2 and implementation roadmap

PART I

Original Specification

Research idea, behavioural framework, solvency maths, regulatory perimeter, architecture and full source code, with reviewer notes inline.

PART II

Review Addendum

28 graded findings, defects reproduced by running the code, algorithm v2, tested reference modules, roadmap, privacy and safety posture.

Edition of 4 October 2026 · Draft for author review

Informational engineering review. Not legal, tax or investment advice.

Contents

Review Summary . . . 3

What this document contains . . . 3

Verdict . . . 3

Priority fixes . . . 3

What is already good . . . 3

Method and limits . . . 4

Part I: Original Specification . . . 5

1. Executive Summary & Problem Formulation . . . 5

2. Behavioral Economics & Psychological Dynamics . . . 6

3. Algorithmic Solvency & Mathematical Framework . . . 7

4. Regulatory Perimeter & Legal Architecture . . . 10

5. Technical Architecture & System Design . . . 11

6. Complete Implementation Source Code . . . 12

7. Step-by-Step Deployment & Operational Manual . . . 23

8. Consumer Stress Scenarios & Verification . . . 24

9. How to Export This Specification to PDF . . . 25

Part II: Review Addendum and v2 Plan . . . 26

A. Findings Register . . . 26

B. Evidence: Defects Reproduced by Running the Shipped Code . . . 28

C. Algorithm v2 Specification . . . 29

D. Reference Implementation (Executed and Tested) . . . 32

E. Implementation Plan v2 . . . 36

F. Privacy, Regulatory & Safety Posture v2 . . . 38

G. Recomputed Verification (v2) and Golden Fixtures . . . 39

Review Summary

What this document contains

Part I is your original specification converted to PDF with text, tables, diagrams, formulas and full source code intact; amber **Review note** boxes carry finding IDs (F-01 to F-28). **Part II** is the review addendum: graded findings, evidence from running the shipped code, a v2 algorithm, tested reference modules, a roadmap, privacy and safety posture, and a recomputed verification table.

Verdict

The concept is sound and differentiated: a conflict-free, deterministic, no-account solvency check with explicit tiers, a pure-function core, and an offline dashboard. The weaknesses are in execution and in a few claims, not in the idea. The review found **28 issues: 3 critical, 10 high, 12 medium, 3 low**. The ones that matter most change what a user is told without any visible error:

- **Runway is ignored** when assigning a tier, so a user with almost no cash buffer can be rated "Solvent".
- **Credit utilization is fabricated** (always 71.4% for anyone with revolving debt).
- **The payoff simulator drops freed minimum payments**, overstating timelines and the Avalanche advantage, and mishandles loans whose payment is below the monthly interest.
- **Amounts and expenses are mis-parsed** ("60k" becomes 60; "1.5 lakh" becomes 1.5; medicine and rent are labelled "cut targets").
- **The zero-PII claim is contradicted by the design**, and the WhatsApp library chosen is unofficial, with account-restriction risk.

Priority fixes

#	Fix	Effort	Findings
1	Pick the channel first: official API for anything public; self-chat allow-list for a personal prototype	Medium	F-14
2	Reframe privacy as data minimisation; add consent notice, retention limit, erase command, pseudonymised IDs	Medium	F-13
3	Add runway and an explicit combination rule; make tier bands half-open	Small	F-01, F-02
4	Ask for credit limits; delete the 1.4x assumption	Small	F-05
5	Fix the payoff engine (roll-over, cascade, negative-amortization guard)	Small	F-06
6	Strict amount parsing with echo confirmation	Small	F-16
7	Rebuild the expense categoriser (word boundaries, essentials, "Uncategorised")	Small	F-17
8	One shared engine module for bot and dashboard; fix the dashboard name-field bug	Medium	F-03, F-22
9	Safety protocol and tone guardrails; verify lenders before advising complaints	Medium	F-25
10	Golden fixtures, property tests and CI	Medium	F-24

What is already good

Pure, I/O-free calculation functions; a clear separation between engine, parser, storage and channel; an offline-first dashboard; a firm no-referral stance; a clear tier concept that is easy to explain; and arithmetic in the FOIR and DCF examples that is correct.

Method and limits

The shipped JavaScript and the v2 modules in section D were executed under Node 22; the patches in D.4 to D.6 and the roadmap are illustrative and were not run. Regulatory and pricing points were checked against public sources in October 2026 and may change; verify them and take legal advice before launch. Nothing here is legal, tax or investment advice.

Part I: Original Specification

Converted from the author's markdown. Text, tables, diagrams, formulas and source code are unchanged. Amber boxes are reviewer annotations tagged with finding IDs from the register in Part II.

1. Executive Summary & Problem Formulation

1.1 The Retail Debt Crisis & Digital Credit Proliferation

Across developing and mature credit markets, retail borrowers face compounding balance-sheet fragility driven by the democratization of unsecured credit lines:

- Instant personal loan apps with opaque fee structures.
- High-velocity Buy-Now-Pay-Later (BNPL) checkout integrations.
- Compounding revolving credit card balances charging 36% to 48% annualized percentage rates (APR).

In markets such as India, unsecured credit growth prompted regulatory authorities—including the Reserve Bank of India (RBI)—to enforce heightened risk weights on retail consumer credit. This regulatory shift highlights systemic concerns over debt stacking, where borrowers leverage short-term digital credit lines to service prior amortizing obligations.



1.2 Structural Market Failures in Personal Finance Management (PFM)

Commercially available personal finance utilities fail to resolve severe indebtedness due to structural misalignments:

Category	Representative Platforms	Business & Monetization Model	Inherent Structural Conflict
Conversational Roast Coaches	Cleo, Candor	Freemium SaaS, proprietary micro-advances	Softens clinical insolvency into humor; monetizes high-cost credit products.
Passive Aggregators	YNAB, Monarch, INDmoney	Referral commissions for loans, credit cards, balance transfers	Requires invasive bank/SMS syncing; avoids declaring absolute insolvency to protect partner loan referral pipelines.
Commercial Debt Settlement	FREED, SingleDebt	Upfront retainers, recurring administrative cuts	Operates downstream of default; incurs high fees, triggers aggressive recovery harassment, and permanently destroys credit scores.
Basic Calculators	Undebt.it, Unburden	Passive utilities, local calculators	Lacks prescriptive operational directives regarding career viability, borrowing freezes, or household dependency structures.

1.3 The Core Solution: An Independent Solvency Diagnostic Engine

The **Debt Reality Checker** operates as an uncompromising, conflict-free diagnostic engine built upon three operating principles:

1. **Absolute Commercial Independence:** Complete absence of credit distribution, loan broking, balance transfer referrals, or financial product affiliate monetization.
2. **Deterministic Mathematical Triage:** Eliminates generative language model hallucinations and sugarcoated advice in favor of deterministic formulas that categorize solvency into binding operational tiers.
3. **Zero-PII Architecture:** Collects zero Personally Identifiable Information (no names, phone identity links, bank credentials, or tax IDs), guaranteeing regulatory exemption under data privacy laws.

[F-09, F-13, F-27] Review note. "Deterministic" removes run-to-run variance, not error: the verdicts are only as good as the thresholds, and those are unvalidated rules of thumb (F-09). Principle 3 is contradicted by the implementation, which stores the WhatsApp ID (a phone number) permanently, and "guaranteeing exemption" is an over-claim (F-13). A "no monetization"

product also needs a funding and validation plan (F-27). Fixes are in Part II.

2. Behavioral Economics & Psychological Dynamics

2.1 The Ostrich Effect in Overindebted Households

Mathematical ignorance is rarely the root driver of compounding consumer debt distress. Instead, psychological avoidance dictates financial behavior.

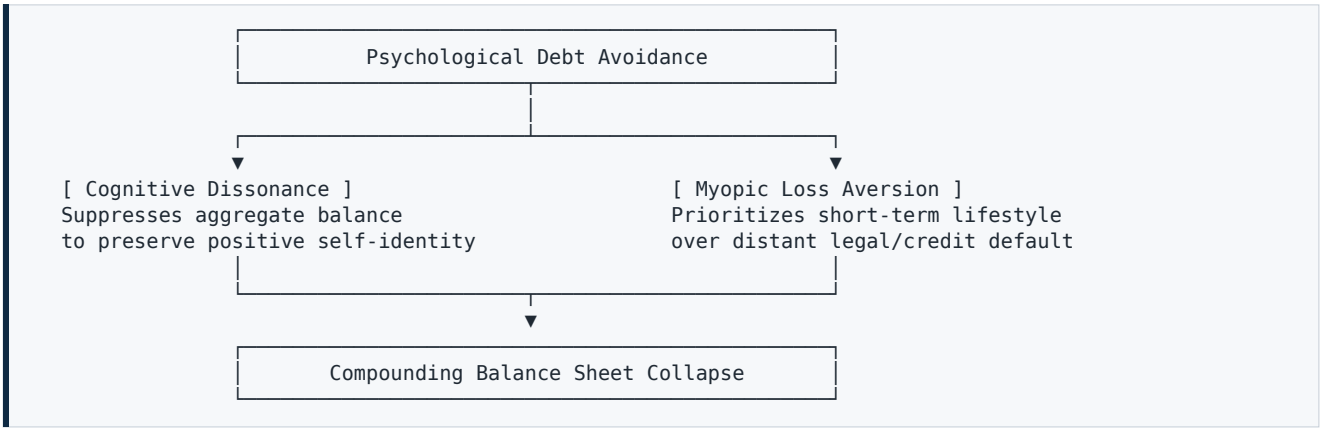
Formalized by Galai and Sade (2003) and expanded by Karlsson, Loewenstein, and Seppi (2009), the **Ostrich Effect** describes how economic agents actively avoid exposing themselves to negative financial feedback:

- Borrowers deliberately stop opening monthly loan and credit card statements.
- Individuals mute or ignore automated SMS transaction alerts from lenders.
- Borrowers avoid computing their aggregate liabilities, maintaining an illusion of manageable balance sheets.

2.2 Cognitive Dissonance & Myopic Loss Aversion

This psychological avoidance is reinforced by two distinct behavioral mechanisms:

- **Cognitive Dissonance:** Acknowledging that fixed obligations exceed monthly cash generation directly threatens the individual's self-image and perceived social status. To avoid mental distress, the mind suppresses the mathematical reality.
- **Myopic Loss Aversion:** The psychological pain of implementing immediate domestic austerity (eliminating discretionary spending, liquidating assets, taking secondary shifts) is acute and present. Conversely, the catastrophic consequences of default (asset repossession, legal recovery, social humiliation) remain temporally distant. Borrowers prioritize immediate emotional relief over long-term balance-sheet survival.



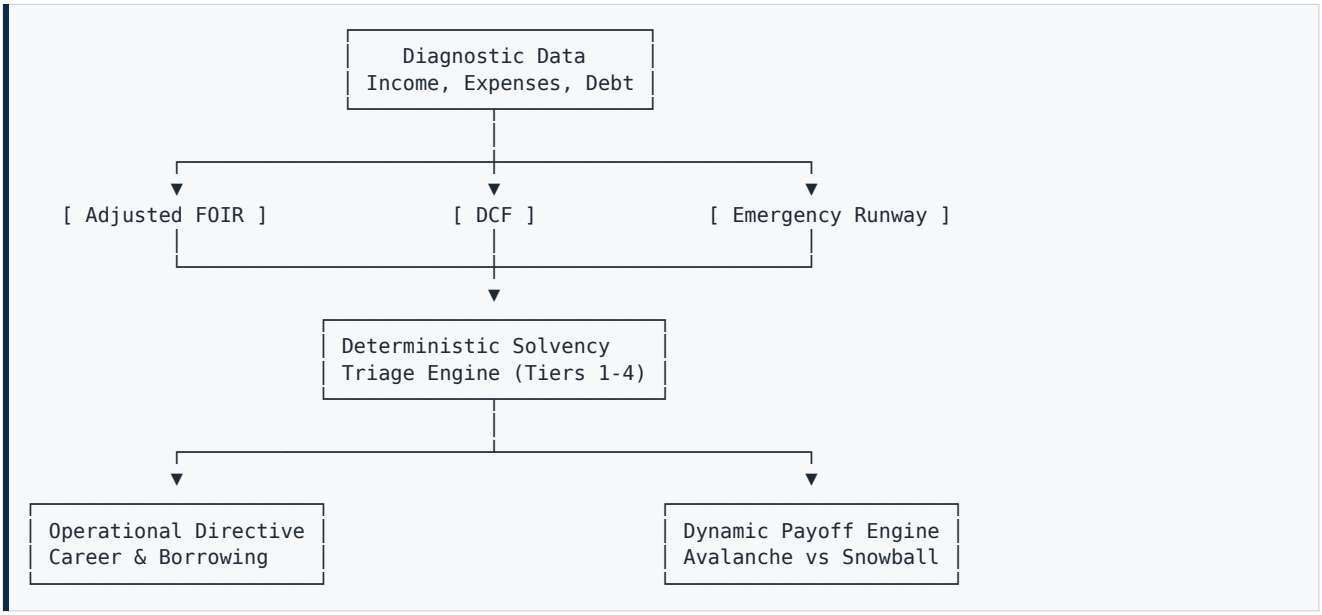
2.3 Radical Candor vs. Punitive Shaming

Traditional financial management tools attempt to solve avoidance by softening outcomes with euphemistic warnings (e.g., "*You went a little over budget this month!*"), which reinforces debt denial. Conversely, hostile or punitive shaming triggers defensiveness, resulting in app uninstalls.

The diagnostic engine implements **Clinical Radical Candor**: an unvarnished audit delivering objective, mathematically backed verdicts that treat personal insolvency as an operational emergency requiring clinical triage.

[F-25, F-26] Review note. Candor is a defensible design goal, but absolute labels ("insolvency", "prohibited", "strictly mandated") delivered with no distress protocol can harm the people the product targets; debt distress is associated with serious mental-health risk. Add tone guardrails, a crisis-escalation path and clear "not legal or financial advice" framing. Citation hygiene: Galai & Sade circulated in 2003 but was published in 2006 (Journal of Business), and the ostrich-effect evidence comes from investors checking portfolios, so applying it to household debt is an extrapolation that should be labelled as such.

3. Algorithmic Solvency & Mathematical Framework



The platform executes pure mathematical evaluations without probabilistic language model inferences.

3.1 Mathematical Formulations

Adjusted Fixed Obligation to Income Ratio ($FOIR_{adjusted}$)

Measures the proportion of net income consumed by mandatory fixed obligations, combining all formal debt servicing with unavoidable housing costs:

$$FOIR_{\text{adjusted}} = \left(\frac{\sum EMI_{\text{all}} + Rent_{\text{mandatory}}}{I_{\text{net}}} \right) \times 100$$

Where:

- Σ EMI_{all} represents all amortized loan EMIs and credit card minimum payments.
- $Rent_{mandatory}$ represents non-negotiable contractual housing rent.
- I_{net} represents net take-home monthly earnings.

Discretionary Cash Flow (DCF)

Computes net operational liquidity remaining after accounting for baseline physical survival and fixed non-negotiable debts:

$$DCF = I_{\text{net}} - \left(E_{\text{core}} + \sum EMI_{\text{all}} + Rent_{\text{mandatory}} \right)$$

Where:

- E_{core} represents survival-line nutritional, medical, and utility expenditures.

Dependency-Adjusted Solvency Index (DASI)

Adjusts cash flow surplus by household demographic load and verified external family support structures:

$$DASI = \left(\frac{DCF}{I_{\text{net}}} \right) \times \left(\frac{N_{\text{earners}}}{N_{\text{earners}} + N_{\text{dep}}} \right) \times (1 + \beta \cdot H_{\text{ext}})$$

Where:

- $N_{earners}$ is the count of active income-generating individuals.
- N_{dep} is the count of non-earning family dependents.
- $H_{ext} \in \{0, 1\}$ denotes external family liquidity support.
- β is a conservative dampening coefficient set to 0.5.

[F-04, F-08] Review note. DASI is not implemented anywhere: the engine accepts *dependents*, *earners* and *familyHelp* but never reads them, and the bot never asks for earners or family help. The formula is also unsound for deficits: when DCF is negative, a larger dependent count shrinks the multiplier and makes the index look *less* negative (better), and family support multiplies a negative number by 1.5. It does not appear in the tier table either. See Part II section C.5 for the replacement. Separately, FOIR counts only minimum payments, so a heavy revolver with a small minimum looks healthier than the balance justifies (F-08).

Emergency Runway (ER)

Determines operational survival time in months if current earnings drop to zero immediately:

$$ER = \frac{\text{Liquid Cash Reserves}}{E_{\text{core}} + Rent_{\text{mandatory}} + \sum EMI_{\text{all}}}$$

3.2 Deterministic Triage Rules & Operational Directives

Diagnostic Metric	Tier 1: Solvent	Tier 2: Fragile / Borderline	Tier 3: Critical Debt Trap	Tier 4: Acute Insolvency
Adjusted FOIR	< 40%	40% to 50%	50% to 65%	> 65%
Discretionary Cash Flow	> 25% of income	10% to 24% of income	0% to 9% of income	≤ 0 (Operational Deficit)
Emergency Runway	≥ 6.0 months	3.0 to 5.9 months	1.0 to 2.9 months	< 1.0 month
Borrowing Directive	Permitted within limits	Mandatory freeze on unsecured lines	Absolute borrowing freeze	Absolute borrowing freeze
Career Risk Policy	Mobility permitted	Salaried retention strongly preferred	Mandatory salaried retention	Mandatory salaried lock-in / extra shifts
Prescribed Action	Surplus investment	Snowball / consolidation paydown	Austerity; Debt Avalanche paydown	Formal lender restructuring

[F-01, F-02, F-10] Review note. The table gives three metrics but no rule for combining them. The code uses FOIR and DCF and ignores runway entirely, so a user with healthy cash flow and 0.4 months of runway is rated Tier 1 with "career mobility permitted" (reproduced in Part II section B). Band edges also overlap (50% appears in both Tier 2 and Tier 3) or leave gaps (24 to 25% DCF; 5.9 to 6.0 months). "Surplus investment" for Tier 1 contradicts section 4.1, which promises the product never recommends investments.

The Career Risk Lock

Distressed borrowers often attempt to launch entrepreneurial ventures or transition into variable commission-only roles in an effort to generate rapid cash injections. However, business cash flows are volatile and lag expectations, whereas retail debt obligations are fixed and compounding.

The engine enforces a deterministic rule: if $FOIR_{adjusted}$ > 40% or ER < 6 months, **entrepreneurial risk is prohibited**. The user is instructed to maintain secure salaried employment and pursue overtime or guaranteed wage shifts until balance-sheet metrics recover to Tier 1 thresholds.

[F-11] Review note. The rule is really about income *volatility*, not entrepreneurship: as written it would also block a move to a higher fixed salary, and the engine cannot "prohibit" anything. Part II section C.7 restates it as an income-variance test with advisory wording.

3.3 Debt Payoff Mechanics: Avalanche vs. Snowball Optimization

The engine models two distinct repayment dynamics:

1. **Closed-End Amortized Loans (Personal Loans, Vehicle Financing):**

$$\text{EMI} = P \times \frac{r(1+r)^n}{(1+r)^n - 1}$$

Where P is principal, r is monthly interest rate, and n is total months.

2. **Revolving Credit Card & Digital Lending Lines:** Compounding runs continuously across 30-day cycles:

$$B_{t+1} = B_t \times \left(1 + \frac{\text{APR}}{365}\right)^{30} - \text{Payment}$$

Paying only minimum payments (5% of principal balance) results in multi-decade amortization loops where cumulative interest exceeds principal debt by multiples.

Algorithmic Payoff Strategy Selection

- **Debt Avalanche (Mathematical Optimum):** Allocates all available surplus discretionary cash flow (DCF) toward the debt with the highest APR, while paying baseline minimums across all other lines. This minimizes total interest paid and shortens the debt-free timeline.
- **Debt Snowball (Behavioral Optimum):** Directs surplus cash flow to the liability with the lowest nominal balance, prioritizing rapid balance-sheet clearance to generate behavioral momentum.
- **Triage Rule:** In Tiers 3 and 4, **the Avalanche method is strictly mandated**. The structural cost of compounding high-APR interest eliminates the viability of behavioral concessions.

[F-06, F-07, F-10] Review note. The simulator in section 6.2 does not implement the defining feature of either method: when a debt is cleared, its freed minimum payment must roll into the next target. It also does not detect negative amortization (minimum below monthly interest), and it uses APR/12 monthly compounding rather than the daily compounding above. The Avalanche mandate is not enforced in code, and behavioral research on debt repayment (Gal & McShane 2012; Amar et al. 2011) gives a reason to allow Snowball when it costs little extra. Measured effects are in Part II section B.

3.4 Credit Bureau Risk Modeling (CIBIL / Experian Proxy)

Without executing invasive credit bureau pull requests, the engine derives bureau-equivalent risk indicators:

- **Credit Utilization Ratio (CUR):**

$$CUR = \left(\frac{\sum \text{Revolving Balances}}{\sum \text{Credit Limits}}\right) \times 100$$

CUR > 30% triggers bureau score degradation; CUR > 70% signals severe distress.

- **Multi-Lender Stacking Velocity:** Maintaining 3 or more active unsecured digital credit lines flags immediate high-risk credit-seeking behavior.
- **Delinquency Velocity:** If DCF < 0, the mathematical time remaining until first hard default is:

$$t_{\text{default}} = \frac{\text{Liquid Cash Reserves}}{|DCF|}$$

[F-05, F-12] Review note. The code never asks for credit limits. It assumes limit = 1.4 x balance, so CUR is 71.4% for every user with any revolving balance, just above the 70% "severe distress" line (reproduced in Part II section B). The stack count is *all* debts (including secured loans and the dummy "None" row the bot inserts), not unsecured lines. "Bureau-equivalent" overstates what a proxy can claim; call these credit-stress indicators.

4. Regulatory Perimeter & Legal Architecture

4.1 SEBI (Investment Advisers) Regulations, 2013 (India)

SEBI Regulation 2(1)(l) governs advice concerning securities (stocks, bonds, mutual funds, derivatives). By restricting its entire scope to **liabilities, cash-flow triage, and debt elimination**—without ever recommending capital market investments, funds, or securities—the platform operates completely outside SEBI's licensing remit.

- The platform avoids protected terms, including "*Investment Adviser*", "*Financial Planner*", or "*Wealth Manager*".
- The product is designated as a "*Solvency Diagnostic Engine*" or "*Debt Health Auditor*".

4.2 RBI Framework & Structural Separation

Commercial settlement firms face ongoing regulatory and legal scrutiny when they collect consumer funds into settlement escrow accounts or negotiate settlements directly with lenders. The engine maintains structural independence:

- It does not disburse, intermediate, or broker credit products.
- It does not accept, hold, or escrow user funds.
- It does not act as a formal legal proxy in recovery disputes.

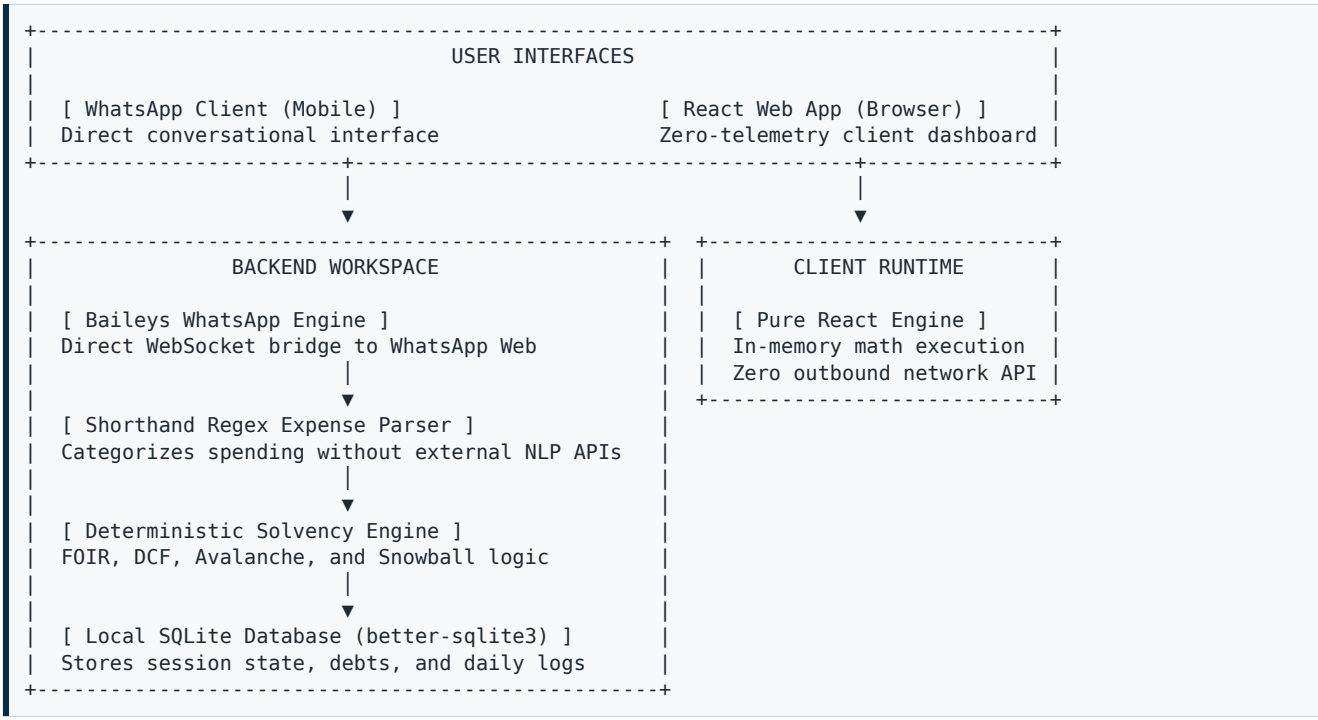
4.3 DPDPA 2023 Compliance via Zero-PII Design

The Digital Personal Data Protection Act (DPDPA), 2023 regulates personal data capable of identifying a natural person. Anonymized numerical figures are exempt from compliance obligations. The platform maintains a zero-PII architecture:

- Zero storage of legal names, email addresses, permanent account numbers (PAN), Aadhaar numbers, or bank account identifiers.
- Phone numbers inside WhatsApp WebSocket sessions are kept strictly in localized, ephemeral database instances without centralized analytics correlation.

[F-13, F-10, F-25] Review note. Anonymized figures may fall outside the Act, but this system is not anonymous: db.js keys every row by the WhatsApp ID, which embeds the phone number, in a permanent file (debt_reality.sqlite), and Baileys keeps session keys on disk. Income and debt figures linked to a phone number are personal data. The DPDP Rules were notified in November 2025 with obligations phased in over roughly 18 months, so treat "exempt" as a claim to retire and design for consent, notice, retention and erasure instead (Part II section F). Take legal advice before launch; this is not legal advice.

5. Technical Architecture & System Design



The system operates entirely via local, zero-cost components:

- **WhatsApp Layer:** Operates over `@whiskeysockets/baileys`, establishing a local WebSocket connection to WhatsApp Web, avoiding Meta Cloud API messaging fees and developer store account charges.
- **Storage Layer:** Uses `better-sqlite3` on local disk, delivering single-file transactional persistence without external database hosting costs.
- **Web Dashboard:** A client-side React single-page application built with Vite that executes all formulas in-browser with zero external API calls.

[F-14, F-23, F-03] Review note. Baileys is an unofficial, reverse-engineered WhatsApp Web client: it is not sanctioned by WhatsApp, accounts can be restricted, and its own documentation says the multi-file auth helper is not meant for production. Two engines (bot and dashboard) already disagree because the logic is duplicated, so the architecture should have one shared core module. "Zero-cost" also ignores uptime (a laptop must stay online) and, as of 1 October 2026, Meta's reported charges for service messages beyond a monthly free allowance on the official API. Verify current pricing before relying on either claim.

6. Complete Implementation Source Code

6.1 Project Setup & Configuration (`package.json`)

```
{
  "name": "debt-reality-suite",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "bot": "node bot.js",
    "web": "vite"
  },
  "dependencies": {
    "@whiskeysockets/baileys": "^6.7.9",
    "better-sqlite3": "^11.8.1",
    "pino": "^9.6.0",
    "qrcode-terminal": "^0.12.0",
    "react": "^18.3.1",
    "react-dom": "^18.3.1"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.3.4",
    "vite": "^6.0.7"
  }
}
```

6.2 Deterministic Solvency & Amortization Engine (`enhanced_engine.js`)

`orig/enhanced_engine.js`

```
/**
 * enhanced_engine.js
 * Deterministic Financial Calculations & Payoff Simulators
 */

export function runFullDiagnostic({
  netIncome,
  coreExpenses,
  rent = 0,
  debts = [],
  liquidReserves = 0,
  dependents = 0,
  earners = 1,
  familyHelp = false
}) {
  const totalMinEmi = debts.reduce((acc, d) => acc + d.minPayment, 0);
  const totalBalance = debts.reduce((acc, d) => acc + d.balance, 0);
  const mandatoryFixed = totalMinEmi + rent;
  const foir = (mandatoryFixed / netIncome) * 100;
  const dcf = netIncome - (coreExpenses + mandatoryFixed);
  const emergencyRunwayMonths = (mandatoryFixed + coreExpenses > 0)
    ? (liquidReserves / (mandatoryFixed + coreExpenses)).toFixed(1)
    : "0.0";

  // Tier classification
  let tier = 1;
  let status = "SOLVENT / CONTROLLED";
  let careerDirective = "Flexible: Career mobility and managed transitions permitted.";
  let borrowingDirective = "Controlled: Borrowing permitted within conservative debt-service caps.";

  if (dcf <= 0 || foir > 65) {
    tier = 4;
    status = "ACUTE INSOLVENCY (OPERATIONAL DEFICIT)";
    careerDirective = "Salaried Lock-in: Starting a business or taking commission-only work is strictly
    ↳ prohibited.";
    borrowingDirective = "ABSOLUTE FREEZE: Do not take new personal loans or credit lines under any
    ↳ circumstances.";
  } else if (foir > 50 || dcf < (0.10 * netIncome)) {
    tier = 3;
    status = "CRITICAL DEBT TRAP";
    careerDirective = "Salaried Retention: Prioritize guaranteed income and overtime. Resignations
    ↳ prohibited.";
    borrowingDirective = "CREDIT FREEZE: Cease all revolving credit card and personal loan drawdowns.";
  } else if (foir > 40 || dcf < (0.25 * netIncome)) {
```

```
    tier = 2;
    status = "FRAGILE / BORDERLINE";
    careerDirective = "Salaried Preferred: Build a 6-month runway before considering job transitions.";
    borrowingDirective = "RESTRICTED: Borrowing permitted strictly for lowering interest rates via structured
    ↳ consolidation.";
  }

  // Credit bureau risk heuristics
  const revolvingDebts = debts.filter(d => d.type === "revolving");
  const revolvingBalance = revolvingDebts.reduce((acc, d) => acc + d.balance, 0);
  const totalLimitBenchmark = revolvingBalance > 0 ? revolvingBalance * 1.4 : 1;
  const curEstimate = revolvingBalance > 0 ? ((revolvingBalance / totalLimitBenchmark) * 100).toFixed(1) :
  ↳ "0.0";

  const bureauRisk = {
    estimatedCUR: `${curEstimate}%`,
    unsecuredStackCount: debts.length,
    defaultRiskLevel: dcf < 0 ? "EXTREME (Default Imminent)" : foir > 50 ? "HIGH" : "MODERATE",
    monthsToDefault: dcf < 0 && liquidReserves > 0
      ? (liquidReserves / Math.abs(dcf)).toFixed(1)
      : (dcf < 0 ? "0.0 (Immediate)" : "N/A (Cash Surplus)")
  };

  // Payoff simulations
  const surplusCash = Math.max(0, dcf);
  const avalanche = simulatePayoff([...debts], surplusCash, "avalanche");
  const snowball = simulatePayoff([...debts], surplusCash, "snowball");

  return {
    tier,
    status,
    foir: foir.toFixed(1),
    dcf: dcf.toFixed(0),
    totalDebt: totalBalance.toFixed(0),
    emergencyRunway: `${emergencyRunwayMonths} months`,
    careerDirective,
    borrowingDirective,
    bureauRisk,
    avalanche,
    snowball,
    interestSavingsWithAvalanche: Math.max(0, snowball.totalInterest - avalanche.totalInterest).toFixed(0)
  };
}

function simulatePayoff(debtList, extraMonthlyCash, strategy = "avalanche") {
  if (debtList.length === 0) {
    return { strategy, monthsToDebtFree: 0, totalInterest: 0, feasible: true };
  }

  let debts = debtList.map(d => ({
    ...d,
    currentBalance: d.balance,
    monthlyRate: (d.apr / 100) / 12
  }));

  let totalInterest = 0;
  let months = 0;
  const maxMonths = 360;

  while (debts.some(d => d.currentBalance > 0) && months < maxMonths) {
    months++;
    let monthlySurplus = extraMonthlyCash;

    for (let debt of debts) {
      if (debt.currentBalance > 0) {
        const interest = debt.currentBalance * debt.monthlyRate;
        totalInterest += interest;
        debt.currentBalance += interest;

        const payment = Math.min(debt.currentBalance, debt.minPayment);
        debt.currentBalance -= payment;
      }
    }

    let activeDebts = debts.filter(d => d.currentBalance > 0);
    if (strategy === "avalanche") {
      activeDebts.sort((a, b) => b.apr - a.apr);
```

```
    } else {
      activeDebts.sort((a, b) => a.currentBalance - b.currentBalance);
    }

    if (activeDebts.length > 0 && monthlySurplus > 0) {
      const target = activeDebts[0];
      const extraPay = Math.min(target.currentBalance, monthlySurplus);
      target.currentBalance -= extraPay;
      monthlySurplus -= extraPay;
    }
  }

  return {
    strategy,
    monthsToDebtFree: months >= maxMonths ? "360+ (Perpetual Trap)" : months,
    totalInterest: Math.round(totalInterest),
    feasible: months < maxMonths
  };
}
```

[F-01, F-03, F-04, F-05, F-06, F-08, F-12] Review note (line-level). (1) The tier block never reads emergencyRunwayMonths. (2) Tier 2 returns "consolidation allowed" while section 3.2 says "mandatory freeze". (3) dependents, earners and familyHelp are accepted and ignored. (4) totalLimitBenchmark = revolvingBalance * 1.4 makes estimatedCUR constant. (5) unsecuredStackCount is debts.length. (6) defaultRiskLevel can never be LOW. (7) simulatePayoff pays each minimum once and applies only the fixed surplus, so freed minimums vanish; if the budget is below monthly interest it runs 360 months and returns an astronomically large totalInterest. (8) dcf <= 0 forces Tier 4 even with large reserves.

6.3 Shorthand Natural Language Expense Parser (`parser.js`)

`orig/parser.js`

```
/**
 * parser.js
 * Offline Regex Expense Extraction
 */

export function parseExpense(text) {
  const trimmed = text.trim();
  const numFirst = /^([0-9]+(?:\.[0-9]{1,2})?)\s+(.+)$/i;
  const numLast = /^(.+)\s+([0-9]+(?:\.[0-9]{1,2})?)$/i;

  let amount = null;
  let description = "";

  if (numFirst.test(trimmed)) {
    const match = trimmed.match(numFirst);
    amount = parseFloat(match[1]);
    description = match[2].trim().toLowerCase();
  } else if (numLast.test(trimmed)) {
    const match = trimmed.match(numLast);
    description = match[1].trim().toLowerCase();
    amount = parseFloat(match[2]);
  } else {
    return null;
  }

  let category = "Discretionary";
  let isEssential = false;

  const essentialGroceries = ["egg", "eggs", "milk", "bread", "dal", "rice", "vegetable", "sabzi", "oil",
  ↳ "ration", "flour", "atta"];
  const transport = ["auto", "metro", "bus", "petrol", "cab", "uber", "ola", "diesel"];
  const habits = ["cigarette", "sutta", "beer", "alcohol", "wine", "vape", "tobacco", "paan"];
  const leisure = ["swiggy", "zomato", "cafe", "coffee", "restaurant", "movie", "netflix", "prime"];

  if (essentialGroceries.some(w => description.includes(w))) {
    category = "Groceries";
    isEssential = true;
  } else if (transport.some(w => description.includes(w))) {
    category = "Commute";
    isEssential = true;
  } else if (habits.some(w => description.includes(w))) {
    category = "Habit / Target to Cut";
```

```
    isEssential = false;
  } else if (leisure.some(w => description.includes(w))) {
    category = "Dining / Leisure";
    isEssential = false;
  }

  return {
    amount,
    description,
    category,
    isEssential
  };
}
```

[F-16, F-17] Review note (line-level). description.includes(w) is substring matching: "cola" contains "ola" (Commute), "price" contains "rice" (Groceries), "business" contains "bus", "automatic" contains "auto". "2 eggs 30" is read as Rs 2 for "eggs 30". Rent, medicine, electricity, school fees and EMIs are not in any list, so they fall through to **"Discretionary"**, which the bot then labels a "Cut Target". Commas, currency symbols and k/lakh suffixes are not parsed, and an unparseable message gets no reply at all.

6.4 SQLite Local Database Persistence Layer (`db.js`)

orig/db.js

```
/**
 * db.js
 * Local SQLite Storage Layer
 */

import Database from "better-sqlite3";

const db = new Database("debt_reality.sqlite");

db.exec(`
  CREATE TABLE IF NOT EXISTS users (
    jid TEXT PRIMARY KEY,
    currency TEXT DEFAULT '₹',
    net_income REAL DEFAULT 0,
    core_expenses REAL DEFAULT 0,
    rent REAL DEFAULT 0,
    liquid_reserves REAL DEFAULT 0,
    dependents INTEGER DEFAULT 0,
    earners INTEGER DEFAULT 1,
    family_help INTEGER DEFAULT 0,
    step INTEGER DEFAULT 0
  );

  CREATE TABLE IF NOT EXISTS debts (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_jid TEXT,
    name TEXT,
    balance REAL,
    apr REAL,
    min_payment REAL,
    type TEXT,
    FOREIGN KEY(user_jid) REFERENCES users(jid)
  );

  CREATE TABLE IF NOT EXISTS expenses (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_jid TEXT,
    amount REAL,
    description TEXT,
    category TEXT,
    is_essential INTEGER,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
  );
`);

export const dbService = {
  getUser(jid) {
    let user = db.prepare("SELECT * FROM users WHERE jid = ?").get(jid);
    if (!user) {
```

```
      db.prepare("INSERT INTO users (jid) VALUES (?)").run(jid);
      user = db.prepare("SELECT * FROM users WHERE jid = ?").get(jid);
    }
    return user;
  },

  updateUser(jid, fields) {
    const keys = Object.keys(fields);
    const setClause = keys.map(k => `${k} = ?`).join(", ");
    const values = Object.values(fields);
    db.prepare(`UPDATE users SET ${setClause} WHERE jid = ?`).run(...values, jid);
  },

  addExpense(jid, expense) {
    db.prepare(`
      INSERT INTO expenses (user_jid, amount, description, category, is_essential)
      VALUES (?, ?, ?, ?, ?)
    `).run(jid, expense.amount, expense.description, expense.category, expense.isEssential ? 1 : 0);
  },

  getDailyExpenses(jid) {
    return db.prepare(`
      SELECT * FROM expenses
      WHERE user_jid = ? AND date(created_at) = date('now', 'localtime')
    `).all(jid);
  },

  addDebt(jid, debt) {
    db.prepare(`
      INSERT INTO debts (user_jid, name, balance, apr, min_payment, type)
      VALUES (?, ?, ?, ?, ?, ?)
    `).run(jid, debt.name, debt.balance, debt.apr, debt.minPayment, debt.type);
  },

  getUserDebts(jid) {
    return db.prepare("SELECT * FROM debts WHERE user_jid = ?").all(jid);
  },

  clearDebts(jid) {
    db.prepare("DELETE FROM debts WHERE user_jid = ?").run(jid);
  }
};
```

[F-13, F-19, F-20] Review note (line-level). (1) jid (phone number) is the primary key of a permanent file. (2) created_at defaults to CURRENT_TIMESTAMP, which is UTC, but getDailyExpenses compares date(created_at) with date('now','localtime'): for IST users, entries between 00:00 and 05:30 land on the previous day's date. Use date(created_at,'localtime'). (3) Money is REAL (floating point); store integer paise. (4) SQLite does not enforce the FOREIGN KEY without PRAGMA foreign_keys = ON; no indexes on user_jid or created_at; no WAL mode. (5) updateUser interpolates column names from object keys; safe today only because every caller passes constants, so whitelist them. (6) clearDebts runs at AUDIT start, destroying prior data before the new audit completes. (7) There is no history table, retention limit or erase command.

6.5 WhatsApp Diagnostic & Logging Client (`bot.js`)

`orig/bot.js`

```
/**
 * bot.js
 * Multi-device WhatsApp Engine with SQLite Persistence
 */

import makeWASocket, { DisconnectReason, useMultiFileAuthState } from "@whiskeysockets/baileys";
import qrcode from "qrcode-terminal";
import pino from "pino";
import { dbService } from "./db.js";
import { parseExpense } from "./parser.js";
import { runFullDiagnostic } from "./enhanced_engine.js";

async function startBot() {
  const { state, saveCreds } = await useMultiFileAuthState("auth_info_baileys");

  const sock = makeWASocket({
    auth: state,
    printQRInTerminal: false,
```

```
  logger: pino({ level: "silent" })
});

sock.ev.on("connection.update", (update) => {
  const { connection, lastDisconnect, qr } = update;
  if (qr) qrcode.generate(qr, { small: true });
  if (connection === "close") {
    const reconnect = lastDisconnect?.error?.output?.statusCode !== DisconnectReason.loggedOut;
    if (reconnect) startBot();
  } else if (connection === "open") {
    console.log("Debt Reality Engine WhatsApp Client is online.");
  }
});

sock.ev.on("creds.update", saveCreds);

sock.ev.on("messages.upsert", async ({ messages }) => {
  const msg = messages[0];
  if (!msg.message || msg.key.fromMe) return;

  const from = msg.key.remoteJid;
  const body = msg.message.conversation || msg.message.extendedTextMessage?.text || "";
  const cleanText = body.trim();
  const user = dbService.getUser(from);
  const curr = user.currency || "₹";

  // Summary command
  if (cleanText.toUpperCase() === "SUMMARY") {
    const logs = dbService.getDailyExpenses(from);
    if (logs.length === 0) {
      await sock.sendMessage(from, { text: "No spending logged today." });
      return;
    }
    const total = logs.reduce((sum, item) => sum + item.amount, 0);
    const essential = logs.filter(i => i.is_essential).reduce((sum, item) => sum + item.amount, 0);
    const discretionary = total - essential;

    const summaryText = `*TODAY'S SPENDING AUDIT*\n` +
      `Total Logged: ${curr}${total.toFixed(0)}\n` +
      `Essential (Needs): ${curr}${essential.toFixed(0)}\n` +
      `Discretionary (Targets to cut): ${curr}${discretionary.toFixed(0)}\n\n` +
      logs.map(l => `• ${l.description}: ${curr}${l.amount} (${l.category})`).join("\n");

    await sock.sendMessage(from, { text: summaryText });
    return;
  }

  // Currency configuration command
  if (cleanText.toUpperCase().startsWith("SET CURRENCY ")) {
    const symbol = cleanText.split(" ")[2] || "₹";
    dbService.updateUser(from, { currency: symbol });
    await sock.sendMessage(from, { text: `Currency updated to ${symbol}` });
    return;
  }

  // Single-line expense parsing
  const parsedExpense = parseExpense(cleanText);
  if (parsedExpense && user.step === 0) {
    dbService.addExpense(from, parsedExpense);
    const reply = `*Logged:* ${curr}${parsedExpense.amount} for *${parsedExpense.description}*\n` +
      `*Category:* ${parsedExpense.category} [${parsedExpense.isEssential ? "Essential" : "Discretionary /
      ↳ Cut Target"}]\n` +
      `_Send *SUMMARY* for today's log or *AUDIT* to run solvency check._`;
    await sock.sendMessage(from, { text: reply });
    return;
  }

  // Diagnostic initiation command
  if (cleanText.toUpperCase() === "AUDIT") {
    dbService.updateUser(from, { step: 1 });
    dbService.clearDebts(from);
    await sock.sendMessage(from, {
      text: `*SOLVENCY AUDIT INITIATED*\nStep 1/5: Enter your net monthly take-home income in ${curr}:`
    });
    return;
  }
```

```
    // Diagnostic state machine
    if (user.step > 0) {
      const numVal = parseFloat(cleanText);
      switch (user.step) {
        case 1:
          if (isNaN(numVal) || numVal <= 0) {
            await sock.sendMessage(from, { text: "Please enter a valid positive number for income." });
            return;
          }
          dbService.updateUser(from, { net_income: numVal, step: 2 });
          await sock.sendMessage(from, { text: `Step 2/5: Enter your monthly essential survival expenses
          ↳ (rent, groceries, medicine) in ${curr}:` });
          break;

        case 2:
          if (isNaN(numVal) || numVal < 0) {
            await sock.sendMessage(from, { text: "Please enter a valid amount." });
            return;
          }
          dbService.updateUser(from, { core_expenses: numVal, step: 3 });
          await sock.sendMessage(from, { text: `Step 3/5: What are your liquid emergency reserves (bank
          ↳ savings, cash) in ${curr}?` });
          break;

        case 3:
          if (isNaN(numVal) || numVal < 0) {
            await sock.sendMessage(from, { text: "Please enter a valid amount." });
            return;
          }
          dbService.updateUser(from, { liquid_reserves: numVal, step: 4 });
          await sock.sendMessage(from, {
            text: `Step 4/5: Add your primary debts.\nSend your primary debt in this format:\n*Name, Balance,
            ↳ APR%, MinMonthlyEMI*\nExample: *CreditCard, 80000, 42, 4000*\n(Or reply *DONE* if finished)`
          });
          break;

        case 4:
          if (cleanText.toUpperCase() === "DONE") {
            const debts = dbService.getUserDebts(from);
            if (debts.length === 0) {
              dbService.addDebt(from, { name: "None", balance: 0, apr: 0, minPayment: 0, type: "installment"
              ↳ });
            }
            dbService.updateUser(from, { step: 5 });
            await sock.sendMessage(from, { text: `Step 5/5: How many non-earning family members depend on this
            ↳ income?` });
            return;
          }

          const parts = cleanText.split(",").map(p => p.trim());
          if (parts.length === 4) {
            dbService.addDebt(from, {
              name: parts[0],
              balance: parseFloat(parts[1]) || 0,
              apr: parseFloat(parts[2]) || 0,
              minPayment: parseFloat(parts[3]) || 0,
              type: parseFloat(parts[2]) > 25 ? "revolving" : "installment"
            });
            await sock.sendMessage(from, { text: `Recorded: ${parts[0]}. Send another debt line or send *DONE*
            ↳ to compute.` });
          } else {
            await sock.sendMessage(from, { text: "Format must be: Name, Balance, APR, MinPayment. Or type
            ↳ *DONE*." });
          }
          break;

        case 5:
          const deps = parseInt(cleanText) || 0;
          dbService.updateUser(from, { dependents: deps, step: 0 });

          const updatedUser = dbService.getUser(from);
          const userDebts = dbService.getUserDebts(from);

          const result = runFullDiagnostic({
            netIncome: updatedUser.net_income,
            coreExpenses: updatedUser.core_expenses,
            liquidReserves: updatedUser.liquid_reserves,
```

```
            dependents: updatedUser.dependents,
            debts: userDebts
          });

          const report = `*============================*\n` +
            `*UNVARNISHED SOLVENCY REPORT*\n` +
            `*============================*\n` +
            `*Status:* ${result.status}\n` +
            `*FOIR:* ${result.foir}% | *Surplus DCF:* ${curr}${result.dcf}/mo\n` +
            `*Emergency Runway:* ${result.emergencyRunway}\n\n` +
            `*CREDIT BUREAU RISK INDICATORS*\n` +
            `• Default Risk: *${result.bureauRisk.defaultRiskLevel}*\n` +
            `• Months to Insolvency: *${result.bureauRisk.monthsToDefault}*\n` +
            `• Active Debt Lines: *${result.bureauRisk.unsecuredStackCount}*\n\n` +
            `*PAYOFF RECOVERY ROADMAP*\n` +
            `• *Avalanche Method:* ${result.avalanche.monthsToDebtFree} mos | Total Interest:
            ↳ ${curr}${result.avalanche.totalInterest}\n` +
            `• *Snowball Method:* ${result.snowball.monthsToDebtFree} mos | Total Interest:
            ↳ ${curr}${result.snowball.totalInterest}\n` +
            `[*] *Avalanche Savings:* You save *${curr}${result.interestSavingsWithAvalanche}* in predatory
            ↳ interest.\n\n` +
            `*OPERATIONAL DIRECTIVES*\n` +
            `• *Career:* ${result.careerDirective}\n` +
            `• *Borrowing:* ${result.borrowingDirective}\n` +
            `*============================*`;

          await sock.sendMessage(from, { text: report });
          break;
      }
    }
  });
}

startBot();
```

Note: the emoji glyph in this listing is shown as [*] because the PDF font has no emoji.

[F-14, F-15, F-16, F-21, F-12] Review note (line-level). (1) messages.upsert reads only messages[0] and ignores the event type, so history-sync batches ("append") are replayed as live messages. (2) No filter on remoteJid: group chats and status@broadcast are processed and each sender gets a database row. (3) On a personal number, every contact who messages you is treated as a user, while msg.key.fromMe makes your own self-chat unusable. (4) startBot() recurses on close with no back-off and without ending the old socket, which can leave duplicate handlers and duplicate replies. (5) parseFloat("60k") is 60, parseFloat("1.5 lakh") is 1.5, parseFloat("50,000") is 50; "|| 0" and parseInt fallbacks zero out bad input silently. (6) type is inferred as "revolving" if APR > 25. (7) No CANCEL: at step > 0 the expense branch is skipped and the user cannot log. (8) Rent, earners and family help are never asked, and the bot passes no rent, so its "FOIR" is EMI / income. (9) The "None" placeholder debt makes a debt-free user show one active line. (10) No first-message consent or disclaimer.

6.6 Zero-Telemetry Client-Side React Dashboard (`SolvencyDashboard.jsx`)

`orig/SolvencyDashboard.jsx`

```
/**
 * SolvencyDashboard.jsx
 * Standalone Client-Side Dashboard
 */

import React, { useState, useMemo } from "react";

export default function SolvencyDashboard() {
  const [income, setIncome] = useState(60000);
  const [coreExpenses, setCoreExpenses] = useState(22000);
  const [liquidCash, setLiquidCash] = useState(30000);
  const [currency, setCurrency] = useState("₹");

  const [debts, setDebts] = useState([
    { id: 1, name: "Credit Card", balance: 65000, apr: 42, minPayment: 3500 },
    { id: 2, name: "Personal Loan", balance: 120000, apr: 16, minPayment: 4500 }
  ]);

  const addDebt = () => {
    setDebts([
      ...debts,
      { id: Date.now(), name: "New Liability", balance: 25000, apr: 24, minPayment: 1500 }
```

```
    ]);
  };

  const updateDebt = (id, field, value) => {
    setDebts(debts.map(d => (d.id === id ? { ...d, [field]: parseFloat(value) || 0 } : d)));
  };

  const removeDebt = (id) => {
    setDebts(debts.filter(d => d.id !== id));
  };

  const analysis = useMemo(() => {
    const totalMinEmi = debts.reduce((sum, d) => sum + d.minPayment, 0);
    const totalDebt = debts.reduce((sum, d) => sum + d.balance, 0);
    const foir = income > 0 ? (totalMinEmi / income) * 100 : 0;
    const dcf = income - (coreExpenses + totalMinEmi);
    const runway = totalMinEmi + coreExpenses > 0 ? (liquidCash / (totalMinEmi + coreExpenses)).toFixed(1) : 0;

    let tier = "Tier 1: Solvent";
    let badgeColor = "#16a34a";
    if (dcf <= 0 || foir > 65) {
      tier = "Tier 4: Acute Insolvency";
      badgeColor = "#dc2626";
    } else if (foir > 50) {
      tier = "Tier 3: Critical Debt Trap";
      badgeColor = "#ea580c";
    } else if (foir > 40) {
      tier = "Tier 2: Fragile / Borderline";
      badgeColor = "#ca8a04";
    }

    const simulate = (strategy) => {
      let active = debts.map(d => ({ ...d, bal: d.balance, rate: d.apr / 100 / 12 }));
      let totalInterest = 0;
      let months = 0;
      const surplus = Math.max(0, dcf);

      while (active.some(d => d.bal > 0) && months < 360) {
        months++;
        active.forEach(d => {
          if (d.bal > 0) {
            const int = d.bal * d.rate;
            totalInterest += int;
            d.bal += int;
            const pay = Math.min(d.bal, d.minPayment);
            d.bal -= pay;
          }
        });

        let unpaid = active.filter(d => d.bal > 0);
        if (strategy === "avalanche") {
          unpaid.sort((a, b) => b.apr - a.apr);
        } else {
          unpaid.sort((a, b) => a.bal - b.bal);
        }

        if (unpaid.length > 0 && surplus > 0) {
          const pay = Math.min(unpaid[0].bal, surplus);
          unpaid[0].bal -= pay;
        }
      }
      return { months, totalInterest: Math.round(totalInterest) };
    };

    const avalanche = simulate("avalanche");
    const snowball = simulate("snowball");

    return { totalMinEmi, totalDebt, foir, dcf, runway, tier, badgeColor, avalanche, snowball };
  }, [income, coreExpenses, liquidCash, debts]);

  return (
    <div style={{ maxWidth: 900, margin: "20px auto", fontFamily: "system-ui, sans-serif", padding: 20, color:
    ↳ "#0f172a" }}>
      <header style={{ borderBottom: "2px solid #e2e8f0", paddingBottom: 15, marginBottom: 20 }}>
        <h1 style={{ margin: 0, fontSize: 24, fontWeight: 800 }}>Deterministic Solvency Auditor</h1>
        <p style={{ margin: "4px 0 0", color: "#64748b", fontSize: 14 }}>Zero-PII • Deterministic Mathematical
        ↳ Simulation • Zero Third-Party Telemetry</p>
```

```
      </header>

      <div style={{ display: "grid", gridTemplateColumns: "repeat(auto-fit, minmax(200px, 1fr))", gap: 15,
      ↳ marginBottom: 20 }}>
        <div style={{ padding: 15, background: "#f8fafc", borderRadius: 8, border: "1px solid #cbd5e1" }}>
          <div style={{ fontSize: 12, color: "#64748b" }}>Solvency Tier</div>
          <div style={{ fontSize: 16, fontWeight: 700, color: analysis.badgeColor, marginTop: 4
          ↳ }}>{analysis.tier}</div>
        </div>
        <div style={{ padding: 15, background: "#f8fafc", borderRadius: 8, border: "1px solid #cbd5e1" }}>
          <div style={{ fontSize: 12, color: "#64748b" }}>Adjusted FOIR</div>
          <div style={{ fontSize: 20, fontWeight: 700, marginTop: 4 }}>{analysis.foir.toFixed(1)}%</div>
        </div>
        <div style={{ padding: 15, background: "#f8fafc", borderRadius: 8, border: "1px solid #cbd5e1" }}>
          <div style={{ fontSize: 12, color: "#64748b" }}>Discretionary Cash Flow (DCF)</div>
          <div style={{ fontSize: 20, fontWeight: 700, color: analysis.dcf <= 0 ? "#dc2626" : "#0f172a",
          ↳ marginTop: 4 }}>
            {currency}{analysis.dcf}
          </div>
        </div>
        <div style={{ padding: 15, background: "#f8fafc", borderRadius: 8, border: "1px solid #cbd5e1" }}>
          <div style={{ fontSize: 12, color: "#64748b" }}>Emergency Runway</div>
          <div style={{ fontSize: 20, fontWeight: 700, marginTop: 4 }}>{analysis.runway} Months</div>
        </div>
      </div>

      <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr 1fr", gap: 15, marginBottom: 20 }}>
        <div>
          <label style={{ fontSize: 12, fontWeight: 600 }}>Monthly Net Income ({currency})</label>
          <input type="number" value={income} onChange={e => setIncome(Number(e.target.value))} style={{
          ↳ width: "100%", padding: 8, marginTop: 4, borderRadius: 6, border: "1px solid #cbd5e1" }} />
        </div>
        <div>
          <label style={{ fontSize: 12, fontWeight: 600 }}>Core Living Costs ({currency})</label>
          <input type="number" value={coreExpenses} onChange={e => setCoreExpenses(Number(e.target.value))}
          ↳ style={{ width: "100%", padding: 8, marginTop: 4, borderRadius: 6, border: "1px solid #cbd5e1" }}
          ↳ />
        </div>
        <div>
          <label style={{ fontSize: 12, fontWeight: 600 }}>Liquid Reserves ({currency})</label>
          <input type="number" value={liquidCash} onChange={e => setLiquidCash(Number(e.target.value))}
          ↳ style={{ width: "100%", padding: 8, marginTop: 4, borderRadius: 6, border: "1px solid #cbd5e1" }}
          ↳ />
        </div>
      </div>

      <h3 style={{ fontSize: 16, marginBottom: 10 }}>Liabilities & Debt Amortization Schedule</h3>
      <table style={{ width: "100%", borderCollapse: "collapse", marginBottom: 15, fontSize: 14 }}>
        <thead>
          <tr style={{ background: "#f1f5f9", textAlign: "left" }}>
            <th style={{ padding: 8 }}>Liability</th>
            <th style={{ padding: 8 }}>Balance ({currency})</th>
            <th style={{ padding: 8 }}>APR (%)</th>
            <th style={{ padding: 8 }}>Min Monthly EMI ({currency})</th>
            <th style={{ padding: 8 }}>Action</th>
          </tr>
        </thead>
        <tbody>
          {debts.map(d => (
            <tr key={d.id} style={{ borderBottom: "1px solid #e2e8f0" }}>
              <td style={{ padding: 8 }}><input type="text" value={d.name} onChange={e => updateDebt(d.id,
              ↳ "name", e.target.value)} style={{ padding: 4, width: 140 }} /></td>
              <td style={{ padding: 8 }}><input type="number" value={d.balance} onChange={e =>
              ↳ updateDebt(d.id, "balance", e.target.value)} style={{ padding: 4, width: 90 }} /></td>
              <td style={{ padding: 8 }}><input type="number" value={d.apr} onChange={e => updateDebt(d.id,
              ↳ "apr", e.target.value)} style={{ padding: 4, width: 60 }} /></td>
              <td style={{ padding: 8 }}><input type="number" value={d.minPayment} onChange={e =>
              ↳ updateDebt(d.id, "minPayment", e.target.value)} style={{ padding: 4, width: 90 }} /></td>
              <td style={{ padding: 8 }}><button onClick={() => removeDebt(d.id)} style={{ color: "#dc2626",
              ↳ border: "none", background: "none", cursor: "pointer", fontWeight: 600 }}>Remove</button></td>
            </tr>
          ))}
        </tbody>
      </table>
      <button onClick={addDebt} style={{ padding: "8px 14px", background: "#f1f5f9", border: "1px solid
      ↳ #cbd5e1", borderRadius: 6, cursor: "pointer", fontSize: 13, marginBottom: 25 }}>+ Add
      ↳ Liability</button>
```

```
      <div style={{ padding: 15, background: "#f8fafc", borderRadius: 8, border: "1px solid #e2e8f0" }}>
        <h4 style={{ margin: "0 0 10px 0", fontSize: 15 }}>Comparative Amortization Payoff Engines</h4>
        <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr", gap: 15 }}>
          <div style={{ padding: 12, background: "#ffffff", borderRadius: 6, border: "1px solid #cbd5e1" }}>
            <div style={{ fontWeight: 700, color: "#0369a1" }}>Debt Avalanche (High Interest First)</div>
            <div style={{ fontSize: 13, marginTop: 4 }}>Payoff Timeline: <b>{analysis.avalanche.months}
            ↳ Months</b></div>
            <div style={{ fontSize: 13 }}>Cumulative Interest:
            ↳ <b>{currency}{analysis.avalanche.totalInterest.toLocaleString()}</b></div>
          </div>
          <div style={{ padding: 12, background: "#ffffff", borderRadius: 6, border: "1px solid #cbd5e1" }}>
            <div style={{ fontWeight: 700, color: "#475569" }}>Debt Snowball (Lowest Balance First)</div>
            <div style={{ fontSize: 13, marginTop: 4 }}>Payoff Timeline: <b>{analysis.snowball.months}
            ↳ Months</b></div>
            <div style={{ fontSize: 13 }}>Cumulative Interest:
            ↳ <b>{currency}{analysis.snowball.totalInterest.toLocaleString()}</b></div>
          </div>
        </div>
        <p style={{ fontSize: 13, color: "#15803d", marginTop: 12, fontWeight: 600 }}>
          The Avalanche strategy reduces wasted compound interest by {currency}{Math.max(0,
          ↳ analysis.snowball.totalInterest - analysis.avalanche.totalInterest).toLocaleString()} compared to
          ↳ Snowball.
        </p>
      </div>
    </div>
  );
}
```

[F-03, F-22, F-28, F-06] Review note (line-level). (1) updateDebt applies parseFloat(value) || 0 to every field, including "name": typing a letter produces NaN, which becomes 0, so the liability name cannot be edited. (2) Tiering uses FOIR only (the engine also uses DCF), so identical inputs can yield different tiers on WhatsApp and the web. (3) No rent input; runway ignored in the tier. (4) setCurrency is never used. (5) Numbers are not formatted with Indian digit grouping (toLocaleString("en-IN")). (6) Labels are not bound to inputs (no htmlFor/id), and tier is conveyed by colour plus text only in one place. (7) The payoff loop inherits the missing roll-over, and shows "360 Months" for infeasible plans. (8) No export or import of scenarios.

6.7 Frontend Scaffold (`index.html`, `main.jsx`, `vite.config.js`)

`index.html`

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Debt Reality Check Engine</title>
  </head>
  <body style="margin: 0; background-color: #f1f5f9;">
    <div id="root"></div>
    <script type="module" src="/main.jsx"></script>
  </body>
</html>
```

`main.jsx`

```
import React from "react";
import ReactDOM from "react-dom/client";
import SolvencyDashboard from "./SolvencyDashboard.jsx";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <SolvencyDashboard />
  </React.StrictMode>
);
```

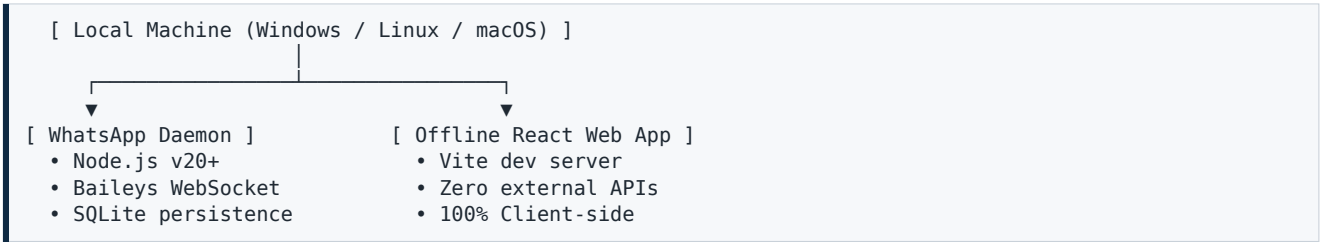
`vite.config.js`

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()]
});
```

---

7. Step-by-Step Deployment & Operational Manual



Step 1: Environment Preparation

Install the Node.js LTS runtime (Node.js v20 or higher) and Git on your local machine:

```
node -v
npm -v
git --version
```

Step 2: Directory Initialization & Installation

Create the working directory and install all required open-source dependencies:

```
mkdir debt-reality-suite
cd debt-reality-suite
```

Save the provided `package.json`, then install dependencies:

```
npm install
```

Step 3: Populate Project Files

Save the implementation files directly in the `debt-reality-suite` root folder:

- `enhanced_engine.js` (Mathematical calculations)
- `parser.js` (Regex-based expense classification)
- `db.js` (SQLite storage layer)
- `bot.js` (WhatsApp Baileys client)
- `SolvencyDashboard.jsx` (React UI dashboard)
- `index.html` (HTML shell)
- `main.jsx` (React entrypoint)
- `vite.config.js` (Vite build configuration)

Step 4: Running the WhatsApp Engine (Zero Infrastructure Cost)

Launch the WhatsApp daemon from your terminal:

```
npm run bot
```

1. An authentication QR code will display directly in your terminal.
2. Open WhatsApp on your mobile phone → Navigate to **Settings → Linked Devices → Link a Device**.
3. Scan the terminal QR code. Authentication credentials will persist locally in the generated `auth_info_baileys` folder.
4. Subsequent launches connect automatically without requiring a new QR scan.

[F-14] Review note. Add auth_info_baileys/ and debt_reality.sqlite to .gitignore before the first commit: the auth folder is equivalent to a logged-in WhatsApp session. Scan with a dedicated number, never your personal one, unless you restrict the bot to your own self-chat (Part II section E).

Step 5: Running the Web Dashboard Locally

In a separate terminal window inside the same folder, launch the Vite development server:

```
npm run web
```

Open `http://localhost:5173` in any standard web browser. All calculations, slider evaluations, and payoff comparisons execute locally inside the browser's JavaScript engine.

8. Consumer Stress Scenarios & Verification

The deterministic engine's triage logic is validated across four household archetypes:

```
[Profile A: Sole Earner] ──────► FOIR: 62.2% ──► Tier 4: Acute Insolvency (Operational Deficit)
[Profile B: Dual Urban]  ──────► FOIR: 50.0% ──► Tier 3: Critical Debt Trap (Avalanche Mandated)
[Profile C: Entrepreneur] ─────► FOIR: 48.1% ──► Tier 2: Fragile (Career Lock: Salaried Retention)
[Profile D: Loan App Trap] ────► FOIR: 109.1% ─► Tier 4: Acute Insolvency (Immediate Intervention)
```

Parameter / Metric	Profile A: Sole Earner	Profile B: Dual-Income Urban	Profile C: Aspiring Founder	Profile D: Predatory App Spiral
Net Income	Rs. 45,000	Rs. 2,00,000 (Household)	Rs. 1,60,000	Rs. 22,000
Total Debt Payments	Rs. 28,000	Rs. 1,00,000	Rs. 77,000	Rs. 24,000
Core Living Costs	Rs. 22,000	Rs. 65,000	Rs. 55,000	Rs. 12,000
Adjusted FOIR	**62.2%**	**50.0%**	**48.1%**	**109.1%**
Discretionary Cash Flow	**-Rs. 5,000**	**+Rs. 35,000**	**+Rs. 28,000**	**-Rs. 14,000**
Emergency Runway	0.2 months	1.1 months	2.27 months	0.0 months
Solvency Tier	**Tier 4: Acute Insolvency**	**Tier 3: Critical Debt Trap**	**Tier 2: Fragile / Locked**	**Tier 4: Acute Insolvency**
Career Directive	Salaried lock; overtime	Salaried retention	Resignation prohibited	Retain hourly wage work
Borrowing Directive	Absolute freeze	Absolute freeze	Restricted to consolidation	Absolute freeze
Action Plan	Immediate debt restructuring	Avalanche paydown on cards	Pay down personal lines	File regulatory complaints

[F-02, F-24, F-25] Review note. The arithmetic (FOIR, DCF) checks out for all four profiles; the tier labels do not follow the stated rules. Run through the shipped engine, Profile B is **Tier 2** (FOIR is exactly 50.0, which is not greater than 50), and Profile C's 2.27-month runway sits in the Tier 3 band, so "Tier 2" is only reachable by ignoring runway while Profile B's Tier 3 is only reachable by using it. Profile D's "File regulatory complaints" is only appropriate after checking whether the lender is an RBI-regulated entity (RBI publishes a public directory of digital lending apps); it should be a conditional branch, not a blanket action. The recomputed table is in Part II section G.

9. How to Export This Specification to PDF

To convert this whitepaper into a PDF document:

1. **Direct Browser Print (Recommended):** Paste this document into any Markdown viewer or previewer (e.g., GitHub, VS Code Markdown Preview, or Obsidian). Press `Ctrl + P` (or `Cmd + P` on macOS). Select **Destination: Save as PDF**. Under Options, enable **Background graphics** and set margins to **Default** or **Minimum**.
2. **VS Code Markdown PDF Extension:** Install the *Markdown PDF* extension (`yzane.markdown-pdf`). Right-click inside the document and select **Markdown PDF: Export (pdf)**.
3. **Pandoc CLI (Terminal Users):**

```
pandoc Debt_Reality_Engine_Spec.md -o Debt_Reality_Engine_Spec.pdf --pdf-engine=xelatex -V geometry:margin=1in
```

Part II: Review Addendum and v2 Plan

Findings, evidence from running the shipped code, the v2 algorithm, reference implementation, roadmap, and safety and privacy posture.

A. Findings Register

Twenty-eight findings, graded by how much each one can change a verdict a user receives, expose user data, or break the product. **Critical** = wrong or unsafe output, or a claim the design contradicts. **High** = materially wrong behaviour in normal use. **Medium** = correctness, robustness or policy gaps. **Low** = evidence, documentation and strategy hygiene. Fixes are specified in sections C to F.

ID	Severity	Area	Finding	Recommended fix
F-01	**High**	Algorithm	Runway is in the tier table but never used; there is no rule for combining the three metrics. Healthy cash flow with 0.4 months of runway is rated Tier 1 with "career mobility permitted".	Explicit combination rule: flow metrics decide, a thin buffer may worsen the tier by at most one level (C.5).
F-02	**High**	Verification	Documented profile tiers do not follow the documented rules or the code (Profile B is Tier 2 in code, Tier 3 in section 8; Profile C's runway is in the Tier 3 band). Band edges overlap at 50% and leave gaps at 24 to 25% DCF and 5.9 to 6.0 months.	Half-open bands; golden fixtures including every boundary (G).
F-03	**High**	Consistency	Bot, engine and dashboard disagree: Tier 2 borrowing is "freeze" in 3.2 but "consolidation allowed" in code; dashboard tiers on FOIR only; bot omits rent, so its "adjusted FOIR" is EMI / income.	One shared core module imported by every surface (E.1).
F-04	**High**	Algorithm	DASI is never implemented and is unsound: for negative DCF, more dependents make the index look better, and family support multiplies a deficit by 1.5.	Retire DASI; dependency-load escalation plus plausibility checks (C.5).
F-05	**Critical**	Algorithm	Credit utilization is fabricated (limit = 1.4 x balance), so CUR is 71.4% for every user with revolving debt, above the 70% "severe" line.	Ask for credit limits; report "unknown" if absent (C.2).
F-06	**High**	Payoff engine	Freed minimums never roll over, leftover surplus does not cascade, negative amortization is undetected. Avalanche advantage overstated about 5x in the test case; an infeasible loan yields a 360-month run and an absurd interest total.	Fixed monthly budget, cascade, BUDGET_BELOW_INTEREST guard (C.6).
F-07	**Medium**	Rates	Spec formula (daily compounding, 5% declining minimum) differs from code (APR / 12, fixed minimum). Flat-rate and up-front-fee loans, the Profile D reality, cannot be entered; "24% flat plus 10% fee over 6 months" is really about 80% APR.	True-APR normaliser (C.3).
F-08	**Medium**	Algorithm	FOIR uses minimum payments only, flattering heavy revolvers. DCF at or below zero forces Tier 4 even with large reserves. "Insolvency" is a legal term.	Add interest burn and months-of-income owed; deficit tiering by months to default; rename Tier 4 (C.4, C.5).
F-09	**Medium**	Calibration	Cut-offs are unvalidated rules of thumb. Flat percentages ignore absolute income, irregular income, household size and self-reported core costs.	Income-history input, plausibility checks, calibration pilot (C.2, C.9, E.4).
F-10	**Medium**	Policy	Tier 1 prescribes "Surplus investment", contradicting the SEBI-perimeter claim. The Avalanche mandate is unenforced and ignores behavioural evidence.	Replace with runway and prepayment; cost-aware strategy selector (C.6, C.8).
F-11	**Medium**	Policy	"Entrepreneurial risk is prohibited" conflates self-employment with income volatility and would block a move to higher fixed pay.	Income-variance rule with advisory wording (C.7).

ID	Severity	Area	Finding	Recommended fix
F-12	**Medium**	Bureau proxy	"Bureau-equivalent" overclaims; stack count includes secured loans and the dummy "None" debt; default risk has no LOW level; debt type is inferred from APR > 25.	Rename "credit-stress indicators"; count unsecured lines with a balance; ask the type.
F-13	**Critical**	Privacy	"Zero-PII" is contradicted by the design: the phone-number-bearing WhatsApp ID is the key of a permanent SQLite file. "Guaranteed exemption" is an over-claim.	Reframe as data minimisation; consent, retention, erase command, pseudonymised IDs (F.1).
F-14	**Critical**	Channel	Baileys is an unofficial WhatsApp Web client (ToS and ban risk; not for production per its docs). Auth folder equals a live session. On a personal number the bot serves every contact but ignores the owner's own messages.	Official API for anything public; self-chat allow-list for a personal prototype; secrets hygiene (E.2).
F-15	**High**	Bot	Reads only messages[0]; ignores event type; answers groups and status broadcasts; creates a DB row per sender; reconnect recursion without back-off; silent on unparseable input.	Loop all messages, filter type and JID, single socket with back-off, help reply (D.4).
F-16	**High**	Input	parseFloat("60k") = 60, ("1.5 lakh") = 1.5, ("50,000") = 50; "or 0" fallbacks zero bad fields silently.	Strict parseAmount plus echo confirmation (D.1).
F-17	**High**	Parser	Substring matching misfiles words; "2 eggs 30" logs Rs 2; medicine, rent, electricity and school fees become "Discretionary / Cut Target".	Word boundaries, essential categories, "Uncategorised" (D.2).
F-18	**Medium**	Product	Logged expenses never feed the audit; core costs are typed from memory.	Pre-fill and cross-check from the last 30 days of logs.
F-19	**Medium**	Data	UTC date compared with local date in getDailyExpenses; IST entries between 00:00 and 05:29 land on the wrong day.	date(created_at, 'localtime') (D.5).
F-20	**Medium**	Data	REAL money; foreign keys off; no WAL or indexes; dynamic column names; clearDebts destroys data early; no history or retention.	Integer paise, pragmas, whitelist, audits table (D.5).
F-21	**Medium**	Bot	No CANCEL or BACK (a user mid-audit cannot log); rent, earners and help never asked; dummy "None" debt; no first-contact notice.	Add commands and notice (D.4, F.3).
F-22	**High**	Dashboard	updateDebt turns the Liability name into 0; tier logic duplicates and diverges; no rent; unused setCurrency; no en-IN formatting; labels unbound; no export.	Fix handler; import the shared engine (D.6).
F-23	**Medium**	Cost / ops	"Zero-cost" ignores uptime. Per a BSP notice, Meta charges service messages beyond 1,000 per number per month from 1 Oct 2026 (Rs 0.145 in India); verify.	Budget; process manager; hosting plan (E.2, E.3).
F-24	**Medium**	Quality	Section 8 figures are hand-calculated; no automated tests or CI. The defects above would have been caught by golden fixtures.	Vitest suite, property tests, CI (E.4).
F-25	**High**	Safety	Absolute labels and "prohibited" language with no distress protocol, disclaimers, or lender-legitimacy check before advising complaints.	Tone guardrails, crisis escalation, RBI directory check (F.3).
F-26	**Low**	Evidence	Galai & Sade is a 2003 working paper published in 2006; ostrich-effect evidence is from investors; debt-repayment studies favour small-balance wins.	Cite precisely; hedge the extrapolation (F.4).
F-27	**Low**	Strategy	No funding model for a no-monetisation product; no success metrics or validation plan.	Pilot plan and sustainability options (E.3, E.5).
F-28	**Low**	Access	English only (many users write Hinglish, Hindi, Bengali); dashboard has unbound labels and colour-led status.	Localised copy; accessibility pass (E.3).

B. Evidence: Defects Reproduced by Running the Shipped Code

`enhanced_engine.js` and `parser.js` were executed unmodified under Node 22. Profile reserves are not stated in section 8, so they were back-solved from the stated runways (A: Rs 10,000; B: Rs 1,81,500; C: Rs 3,00,000; D: nil). The "v2" column is the reference implementation in section D.

B.1 Tier verdicts for the four archetypes

Profile	Section 8 says	Shipped engine returns	v2 returns
A: Sole Earner	Tier 4	Tier 4	Tier 4
B: Dual-Income Urban	Tier 3	**Tier 2** (FOIR is exactly 50.0, not above 50)	Tier 3
C: Aspiring Founder	Tier 2	Tier 2 (runway of 2.27 months ignored)	**Tier 3** (runway-limited)
D: Predatory App Spiral	Tier 4	Tier 4	Tier 4

B.2 Other reproduced defects

Test	Shipped code	v2
Income Rs 1,00,000; core Rs 20,000; EMI Rs 5,000; reserves Rs 10,000	Tier 1 "SOLVENT" with runway 0.4 months and "career mobility permitted"	Tier 2, "runway-limited" (flow tier 1)
Revolving balance of Rs 10,000, Rs 65,000, Rs 3,00,000	estimatedCUR = 71.4% in all three cases	Uses the real limit (Rs 65,000 on a Rs 1,00,000 limit gives 65%); "unknown" if no limit
Card 65k at 42%, loan 120k at 16%, BNPL 15k at 30%; income 60,000, core 40,000	15 months; Avalanche Rs 25,071 vs Snowball Rs 27,731 (gap Rs 2,660)	12 months; Rs 22,147 vs Rs 22,690 (gap Rs 543): the old gap was overstated about 5x
Rs 1,00,000 at 48% with Rs 3,000 minimum; income 23,500, core 20,000	"360+ (Perpetual Trap)" with totalInterest = Rs 16,94,11,98,921	BUDGET_BELOW_INTEREST: interest Rs 4,000 a month against a Rs 3,500 budget
Debt-free user (bot inserts the "None" row)	unsecuredStackCount = 1	0 lines
Amounts "60k", "1.5 lakh", "50,000", "2 lakh"	parseFloat gives 60, 1.5, 50, 2	60,000; 1,50,000; 50,000; 2,00,000; "12abc" is rejected
Liability name typed in dashboard	parseFloat("Credit Card") is NaN, which the handler turns into 0	Name stored as text
Expense logged at 01:00 IST on 5 Oct	Stored as 4 Oct 19:30 UTC; missing from the 5 Oct SUMMARY	Compared in local time

B.3 Expense parser

Message	Shipped parser	v2 parser
2 eggs 30	Rs 2, Groceries	Rs 30, Groceries (asks to confirm)
cola 40	Commute, essential ("ola")	Uncategorised (asks E or D)
price tag 100	Groceries, essential ("rice")	Uncategorised
business lunch 250	Commute, essential ("bus")	Uncategorised
automatic gate repair 800	Commute, essential ("auto")	Uncategorised
medicine 400	Discretionary, cut target	Health, essential
electricity 1500	Discretionary, cut target	Utilities, essential
rent 12000	Discretionary, cut target	Rent & Housing, essential
school fees 3000	Discretionary, cut target	Education & Childcare, essential
milk 1,200	No reply at all	Rs 1,200, Groceries
tea ₹50	No reply at all	Rs 50, Uncategorised

C. Algorithm v2 Specification

C.1 Design principles

1. **Explicit, total, testable.** Every input combination maps to exactly one tier; bands are half-open, so a value on a cut-off belongs to the worse tier.
2. **Never invent data.** Unknown quantities (credit limit, rate basis) are asked for or shown as unknown; nothing is back-filled with a multiplier.
3. **Cost-true.** Every loan is normalised to the rate the borrower really pays, fees included.
4. **Explainable.** Each verdict carries its drivers and the distance to the next better tier.
5. **Pure.** The core never touches the network, clock, database or randomness, so bot, web and tests share one implementation.

C.2 Inputs

Field	Meaning	Change from v1
netIncome, incomeHistory	Monthly take-home; last 3 to 6 months for irregular earners	History added; the engine uses the lowest recent month
rent	Contractual housing cost, entered separately	Was folded into core costs in the bot
coreExpenses	Food, medical, utilities, transport, school fees. **Excludes rent and EMIs**	Definition tightened
liquidReserves	Cash and bank balances accessible within days	Unchanged
earners, dependents	Household composition	Now used (C.5)
debt.type, debt.secured	installment, revolving, bnpl, loanapp; secured yes or no	Asked, not inferred from APR
debt.balance, minPayment	Outstanding principal; actual monthly minimum	Unchanged
debt.apr or quote	Reducing-balance APR, or flat rate, daily rate and up-front fee	Normalised by C.3
debt.creditLimit	Required for revolving lines	New; replaces the 1.4x assumption

C.3 Rate normalisation (what a loan really costs)

Loan apps quote flat or daily rates and deduct fees up front. Convert every quote to one comparable figure. For a flat rate f (as a decimal per year) on principal P over n months:

$$EMI_{\text{flat}} = \frac{P(1 + fn/12)}{n}$$

The true monthly rate r solves the equation below (fee F deducted from the cash received); it is found by bisection, and the nominal annual figure is 1200 r:

$$P - F = EMI \times \frac{1 - (1+r)^{-n}}{r} \qquad \Rightarrow \qquad APR_{\text{true}} = 1200\, r$$

A daily rate d (in percent) converts to a nominal APR of 365 d. **Worked example (executed):** a Rs 20,000 loan at "24% flat", 6 months, has EMI Rs 3,733.33 and a true APR of **40.0%**; add a 10% up-front fee (Rs 2,000) and the true APR becomes **79.6%**.

C.4 Metrics

Effective income for irregular earners uses the weakest recent month, optionally stressed by a haircut s (0 by default; 0.1 to 0.2 for a stress run):

$$I_{\text{eff}} = \min(I_{\text{net}},\ I_{t-1},\ \ldots,\ I_{t-k}) \times (1 - s)$$

FOIR, DCF and runway keep the original definitions, evaluated on I_{eff}. Two additions address the minimum-payment blind spot (F-08):

$$IBR = \sum_i B_i \cdot \frac{APR_i}{1200} \qquad DTI_{\text{months}} = \frac{\sum_i B_i}{I_{\text{eff}}}$$

IBR is the rupees of interest accruing each month; DTI is how many months of income are owed. When DCF is negative, the months to default are the reserves divided by the absolute deficit, as in the original.

C.5 Tier assignment

Metric	Tier 1	Tier 2	Tier 3	Tier 4
FOIR (% of income)	below 40	40 up to 50	50 up to 65	65 and above
DCF (% of income)	25 and above	10 up to 25	0 up to 10	below 0 and reserves last under 6 months (below 0 with 6+ months of reserves is Tier 3)
Emergency runway (months)	6 and above	3 up to 6	1 up to 3	below 1

Flow metrics (FOIR, DCF) decide the tier. The cash buffer can worsen it by at most one level, because healthy cash flow can rebuild a buffer but a stock of cash cannot rescue a structural deficit:

$$T_{\text{flow}} = \max(T_{FOIR},\ T_{DCF}) \qquad T_{\text{final}} = \max(T_{\text{flow}},\ \min(T_{ER},\ T_{\text{flow}} + 1))$$

Two overrides apply afterwards: (1) any debt whose minimum is at or below its monthly interest forces at least Tier 3 (the balance is growing); (2) **dependency escalation** replaces DASI: if dependents / (earners + dependents) is 0.5 or more and runway is under 3 months, the tier rises by one (capped at 4). External family help is displayed as a contingent buffer but never improves the tier, because it cannot be verified and may not be available. Tier 4 is renamed **"Acute cash-flow crisis"**; "insolvency" has a legal meaning the tool cannot assess.

Distance to the next tier makes verdicts actionable. With M the mandatory payments and c the next better FOIR cut-off, the reduction needed is:

$$\Delta_{FOIR} = M - \frac{c}{100} I_{\text{eff}}$$

and equivalent expressions exist for DCF and runway. The report states the driver metrics and the smallest change that would improve the tier.

C.6 Payoff engine and strategy selection

The monthly debt-service budget is fixed at the sum of minimums plus positive DCF. Each month: accrue interest, pay each minimum in priority order, then cascade everything left down the priority list. Because the budget never shrinks, a cleared debt's minimum rolls into the next target automatically, which is the defining behaviour of both methods. If the budget is at or below the monthly interest the plan is reported as BUDGET_BELOW_INTEREST (with the shortfall) instead of simulating to the horizon.

Avalanche remains the cost-minimising order. To honour the behavioural evidence without abandoning cost discipline, the selector recommends Snowball only when the borrower is in Tier 1 or 2 and the extra interest is small. It always shows the rupee cost of the choice:

$$\Delta I = I_{\text{snowball}} - I_{\text{avalanche}} \qquad \text{Snowball iff } \; T \le 2 \; \wedge \; \Delta I \le 0.01 \sum_i B_i$$

The 1% threshold is a tunable policy value to be set from pilot data.

C.7 Career-risk rule (restated)

Advise against moves that add income variance or reduce fixed monthly pay (commission-only roles, self-employment, unpaid gaps) while FOIR is 40% or more or runway is under 6 months. A move to equal or higher fixed pay is acceptable provided runway stays at 3 months or more through any notice period. The wording is advisory ("not advised until..."), not prohibitive.

C.8 Directives by tier

Tier	Label	Borrowing	Career	Next 30 days
1	Solvent	Permitted while FOIR stays below 40%	Mobility permitted; keep runway at 6 months or more	Build runway to 6 months; prepay the highest-APR debt; for investing, consult a SEBI-registered adviser (outside this tool)
2	Fragile	No new unsecured debt; refinance only if the true APR falls after all fees	Prefer fixed pay; move only to equal or higher fixed pay	Avalanche, or Snowball if C.6 allows; build runway
3	Critical debt trap	Freeze: no new credit lines, no balance-transfer churn	Keep fixed income; avoid variable-pay moves	Avalanche; trim discretionary spend; ask lenders for hardship terms if DCF stays under 10%
4	Acute cash-flow crisis	Freeze	Protect income; add fixed-pay hours if possible	Contact lenders before missing a payment; verify each lending app in the RBI directory; seek non-profit credit counselling; document and report harassment

C.9 Data-quality guards

- Echo every amount with Indian digit grouping and ask for confirmation ("You entered Rs 60,000 per month, correct?") to catch 10x slips.
- Ask "per year or per month?" when a quoted rate exceeds 100.
- Flag implausible inputs: core costs below 10% of income, total EMIs above income, any single amount above 10x monthly income, reserves above 5 years of expenses.
- Reject unknown suffixes rather than guessing; never default a failed parse to zero.

D. Reference Implementation (Executed and Tested)

These three modules were run under Node 22 against the cases in section B and produce the v2 figures quoted there. They are pure ES modules with no dependencies. Snippets in D.4 to D.6 are illustrative patches and were **not** executed.

D.1 `input.js`: strict amount parsing

v2/input.js

```
/**
 * input.js  (v2)
 * Strict amount parsing. Never guesses: an unknown suffix or stray text returns null.
 */
const MULT = { k: 1e3, thousand: 1e3, l: 1e5, lac: 1e5, lakh: 1e5, lakhs: 1e5, cr: 1e7, crore: 1e7, crores:
↳ 1e7 };

export function parseAmount(raw) {
  if (raw == null) return null;
  const s = String(raw).toLowerCase()
    .replace(/₹|inr|rs\.?/g, "")      // currency markers
    .replace(/,/g, "")                // thousands separators
    .trim();
  const m = s.match(/^(\d+(?:\.\d+)?)\s*([a-z]+)?$/);
  if (!m) return null;
  const n = parseFloat(m[1]);
  if (!m[2]) return n;
  return MULT[m[2]] ? n * MULT[m[2]] : null;
}

/** Echo-confirm helper: formats with Indian digit grouping so the user can spot a 10x slip. */
export const inr = (n) => "₹" + Math.round(n).toLocaleString("en-IN");
```

D.2 `expense_parser.js`: word-boundary categorisation

v2/expense_parser.js

```
/**
 * expense_parser.js  (v2)
 * Word-boundary categorisation; unknown spend is "Uncategorised", never silently "cut target".
 */
import { parseAmount } from "./input.js";

const RULES = [
  { cat: "Rent & Housing",        essential: true,  re: /\b(rent|maintenance|society)\b/ },
  { cat: "Debt Service (EMI)",    essential: true,  re: /\b(emi|instalment|installment|loan)\b/ },
  { cat: "Health",                essential: true,  re: /\b(medicines?|pharmacy|doctor|clinic|hospital)\b/ },
  { cat: "Utilities",             essential: true,  re: /\b(electricity|bijli|water|gas|lpg|recharge|broadband|
  ↳ wifi)\b/ },
  { cat: "Education & Childcare", essential: true,  re: /\b(school|tuition|daycare)\b/ },
  { cat: "Groceries",             essential: true,  re: /\b(eggs?|milk|bread|dal|rice|vegetables?|sabzi|oil|rat
  ↳ ion|flour|atta)\b/ },
  { cat: "Commute",               essential: true,  re: /\b(auto|metro|bus|petrol|diesel|train)\b/ },
  { cat: "Cab / Ride-hail",       essential: false, re: /\b(cab|uber|ola|rapido)\b/ },
  { cat: "Habit / Review",        essential: false, re: /\b(cigarettes?|sutta|beer|alcohol|wine|vape|tobacco|pa
  ↳ an)\b/ },
  { cat: "Dining / Leisure",      essential: false, re: /\b(swiggy|zomato|cafe|coffee|restaurant|movie|netflix|
  ↳ prime|chai)\b/ },
];

export function parseExpense(text) {
  // "1.5 lakh" -> "1.5lakh"; "₹ 500" / "rs. 500" -> "500"
  const t = text.trim().toLowerCase()
    .replace(/(₹|rs\.?)\s*(?=\d)/g, "")
    .replace(/(\d)\s+(k|l|lac|lakhs?|cr|crores?)\b/g, "$1$2");
  const tokens = t.split(/\s+/);
  const numeric = tokens.map((w, i) => (parseAmount(w) !== null ? i : -1)).filter(i => i >= 0);
  if (numeric.length === 0) return null;

  // One number -> that's the amount. Several -> the LAST token must be the amount ("2 eggs 30" = Rs 30).
  const last = tokens.length - 1;
  const idx = numeric.length === 1 ? numeric[0] : (numeric.includes(last) ? last : -1);
  if (idx < 0) return { error: "AMBIGUOUS_AMOUNT" };

  const amount = parseAmount(tokens[idx]);
```

```
  const description = tokens.filter((_, i) => i !== idx).join(" ");
  const rule = RULES.find(r => r.re.test(description));
  return {
    amount,
    description,
    category: rule ? rule.cat : "Uncategorised",
    isEssential: rule ? rule.essential : null,      // null = ask the user (reply E or D)
    needsConfirm: numeric.length > 1,
  };
}
```

D.3 `engine_v2.js`: tiers, rate normalisation, payoff and strategy

v2/engine_v2.js

```
/**
 * engine_v2.js
 * Deterministic solvency engine, v2. Pure functions: no I/O, no clock, no randomness.
 * Money is in rupees (use integer paise in storage). Rates are nominal annual % on a reducing balance.
 */

/* ---------- 1. Tier policy (half-open bands: a value ON a cut-off belongs to the worse tier) ---------- */
export const POLICY = {
  foirCuts: [40, 50, 65],       // % of income      -> tiers 1..4 (ascending is worse)
  dcfPctCuts: [25, 10],         // % of income      -> tiers 1..3 for DCF >= 0
  erCuts: [6, 3, 1],            // months of runway -> tiers 1..4 (descending is worse)
  deficitHorizonMonths: 6,      // DCF < 0 and reserves last < 6 months  -> tier 4, else tier 3
  runwayMaxPenalty: 1,          // a thin cash buffer can worsen a flow-based tier by at most one level
  snowballMaxExtraCostPct: 1,   // snowball allowed if it costs <= 1% of total debt more than avalanche
  incomeStress: 0,              // set 0.1-0.2 to run the whole diagnostic on stressed income
};

const tierUp = (v, cuts) => 1 + cuts.filter(c => v >= c).length;     // bigger value = worse
const tierDown = (v, cuts) => 1 + cuts.filter(c => v < c).length;    // smaller value = worse

/* ---------- 2. Rate normalisation: what a loan REALLY costs ---------- */
/** Nominal APR (%/yr, reducing balance) implied by cash actually received vs EMIs actually paid. */
export function trueApr({ principal, fee = 0, emi, months }) {
  const net = principal - fee;
  if (net <= 0 || emi * months <= net) return 0;
  const pv = r => (r === 0 ? emi * months : (emi * (1 - Math.pow(1 + r, -months))) / r);
  let lo = 0, hi = 1;
  while (pv(hi) > net && hi < 1e4) hi *= 2;
  for (let i = 0; i < 200; i++) { const mid = (lo + hi) / 2; if (pv(mid) > net) lo = mid; else hi = mid; }
  return ((lo + hi) / 2) * 12 * 100;
}
export const flatToEmi = ({ principal, flatRatePctPerYear, months }) =>
  (principal + principal * (flatRatePctPerYear / 100) * (months / 12)) / months;
export const dailyRateToApr = (dailyPct) => dailyPct * 365;   // nominal, e.g. 0.1%/day -> 36.5%

/* ---------- 3. Payoff simulation with roll-over of freed minimums ---------- */
export function simulatePayoff(debts, monthlyBudget, strategy = "avalanche", maxMonths = 600) {
  const book = debts.filter(d => d.balance > 0).map(d => ({ ...d, bal: d.balance, r: d.apr / 1200 }));
  if (!book.length) return { strategy, feasible: true, months: 0, totalInterest: 0 };

  const interest0 = book.reduce((s, d) => s + d.bal * d.r, 0);
  if (monthlyBudget <= interest0)
    return { strategy, feasible: false, reason: "BUDGET_BELOW_INTEREST",
             monthlyInterest: Math.round(interest0), shortfall: Math.round(interest0 - monthlyBudget) };

  const order = strategy === "avalanche"
    ? (a, b) => b.apr - a.apr || a.bal - b.bal
    : (a, b) => a.bal - b.bal || b.apr - a.apr;

  let months = 0, totalInterest = 0;
  while (months < maxMonths && book.some(d => d.bal > 0.005)) {
    months++;
    let pool = monthlyBudget;                                  // budget stays constant => freed minimums
    ↳ roll over
    for (const d of book) if (d.bal > 0) { const i = d.bal * d.r; d.bal += i; totalInterest += i; }
    for (const d of book.filter(x => x.bal > 0).sort(order)) {   // minimums, priority order
      const p = Math.min(d.bal, d.minPayment, pool); d.bal -= p; pool -= p;
    }
    for (const d of book.filter(x => x.bal > 0).sort(order)) {   // everything left cascades down the list
      if (pool <= 0) break;
```

```
      const p = Math.min(d.bal, pool); d.bal -= p; pool -= p;
    }
  }
  const left = book.reduce((s, d) => s + d.bal, 0);
  return left > 0.005
    ? { strategy, feasible: false, reason: "EXCEEDS_HORIZON", endingBalance: Math.round(left) }
    : { strategy, feasible: true, months, totalInterest: Math.round(totalInterest) };
}

export function recommendStrategy(tier, av, sn, totalBalance) {
  if (!av.feasible || !sn.feasible) return { pick: "avalanche", why: "Plan is infeasible at current cash flow;
  ↳ restructuring comes first." };
  const delta = sn.totalInterest - av.totalInterest;
  const cheap = delta <= (POLICY.snowballMaxExtraCostPct / 100) * totalBalance;
  return tier <= 2 && cheap
    ? { pick: "snowball", delta, why: "Quick wins cost little extra; momentum improves completion odds." }
    : { pick: "avalanche", delta, why: "Highest-APR-first saves the most interest." };
}

/* ---------- 4. Diagnostic ---------- */
export function runDiagnosticV2(input) {
  const { netIncome, incomeHistory = [], coreExpenses, rent = 0, debts = [], liquidReserves = 0,
          earners = 1, dependents = 0 } = input;

  // Effective income: the lowest of recent months for irregular earners, optionally stressed.
  const incomeEff = Math.min(netIncome, ...incomeHistory) * (1 - POLICY.incomeStress);
  const live = debts.filter(d => d.balance > 0 || d.minPayment > 0);
  const totalEmi = live.reduce((s, d) => s + d.minPayment, 0);
  const totalBalance = live.reduce((s, d) => s + d.balance, 0);
  const mandatory = totalEmi + rent;
  const outflow = coreExpenses + mandatory;

  const foir = (mandatory / incomeEff) * 100;
  const dcf = incomeEff - outflow;
  const dcfPct = (dcf / incomeEff) * 100;
  const er = outflow > 0 ? liquidReserves / outflow : Infinity;
  const interestBurn = live.reduce((s, d) => s + (d.balance * d.apr) / 1200, 0);
  const dtiMonths = totalBalance / incomeEff;                    // months of income owed
  const negAm = live.filter(d => d.balance > 0 && d.minPayment <= (d.balance * d.apr) / 1200).map(d => d.name);
  const unsecuredLines = live.filter(d => !d.secured && d.balance > 0).length;
  const revolving = live.filter(d => d.type === "revolving");
  const cur = revolving.length && revolving.every(d => d.creditLimit > 0)
    ? (revolving.reduce((s, d) => s + d.balance, 0) / revolving.reduce((s, d) => s + d.creditLimit, 0)) * 100
    : null;                                                       // unknown, never invented

  // Per-metric tiers
  const tFoir = tierUp(foir, POLICY.foirCuts);
  const monthsToDefault = dcf < 0 ? liquidReserves / Math.abs(dcf) : Infinity;
  const tDcf = dcf < 0 ? (monthsToDefault < POLICY.deficitHorizonMonths ? 4 : 3)
                       : tierDown(dcfPct, POLICY.dcfPctCuts);
  const tEr = tierDown(er, POLICY.erCuts);

  // Combine: flow metrics decide; the cash buffer may worsen the verdict by at most one tier.
  let tier = Math.max(tFoir, tDcf);
  const flowTier = tier;
  tier = Math.max(flowTier, Math.min(tEr, flowTier + POLICY.runwayMaxPenalty));
  const notes = [];
  if (tier > flowTier) notes.push("Runway-limited: cash flow alone would rate better, but the cash buffer is
  ↳ thin.");
  if (negAm.length && tier < 3) { tier = 3; notes.push(`Balance growing on: ${negAm.join(", ")} (minimum below
  ↳ monthly interest).`); }
  const dlf = dependents / (earners + dependents);
  if (dlf >= 0.5 && er < 3 && tier < 4) { tier += 1; notes.push("Dependency escalation: half or more of the
  ↳ household is non-earning and runway < 3 months."); }

  const drivers = [["FOIR", tFoir], ["DCF", tDcf], ["Runway", tEr]].filter(([, t]) => t === tier).map(([n]) =>
  ↳ n);

  // Payoff: the whole debt-service budget is fixed; freed minimums roll over.
  const budget = totalEmi + Math.max(0, dcf);
  const avalanche = simulatePayoff(live, budget, "avalanche");
  const snowball = simulatePayoff(live, budget, "snowball");

  return { tier, flowTier, tiers: { foir: tFoir, dcf: tDcf, runway: tEr }, drivers, notes,
    metrics: { incomeEff, foir, dcf, dcfPct, er, interestBurn, dtiMonths, cur, unsecuredLines,
    ↳ monthsToDefault, totalBalance },
```

```
    careerRiskLock: foir >= POLICY.foirCuts[0] || er < 6,
    avalanche, snowball, strategy: recommendStrategy(tier, avalanche, snowball, totalBalance) };
}
```

D.4 Bot hardening (`bot.js`, illustrative)

```
// Illustrative patch: not executed.
const ALLOWED = new Set((process.env.ALLOWED_JIDS || "").split(",").filter(Boolean)); // consented users only
let sock, retry = 0;

async function startBot() {
  const { state, saveCreds } = await useMultiFileAuthState("auth_info_baileys"); // prototype only
  sock?.ev.removeAllListeners();            // never keep two handler sets alive
  sock = makeWASocket({ auth: state, logger: pino({ level: "silent" }) });

  sock.ev.on("connection.update", ({ connection, lastDisconnect, qr }) => {
    if (qr) qrcode.generate(qr, { small: true });
    if (connection === "open") retry = 0;
    if (connection === "close") {
      const code = lastDisconnect?.error?.output?.statusCode;
      if (code !== DisconnectReason.loggedOut)
        setTimeout(startBot, Math.min(30000, 1000 * 2 ** retry++));   // exponential back-off
    }
  });
  sock.ev.on("creds.update", saveCreds);

  sock.ev.on("messages.upsert", async ({ messages, type }) => {
    if (type !== "notify") return;                                    // skip history sync
    for (const msg of messages) {                                     // all messages, not just [0]
      const jid = msg.key.remoteJid;
      if (!msg.message || jid.endsWith("@g.us") || jid === "status@broadcast") continue;
      if (!ALLOWED.has(jid)) continue;                                // consent gate / allow-list
      // Personal self-chat prototype instead: accept only msg.key.fromMe in your own "message yourself" chat.
      const text = (msg.message.conversation || msg.message.extendedTextMessage?.text || "").trim();
      await route(jid, text);   // shared router: CANCEL, BACK, HELP, SUMMARY, AUDIT, DELETE MY DATA, expense
    }
  });
}
```

D.5 Storage hardening (`db.js`, illustrative)

```
// Illustrative patch: not executed.
db.pragma("journal_mode = WAL");
db.pragma("foreign_keys = ON");
db.exec(`
  CREATE INDEX IF NOT EXISTS idx_exp_user_time ON expenses(user_jid, created_at);
  CREATE TABLE IF NOT EXISTS audits (            -- trend history: the retention feature
    id INTEGER PRIMARY KEY, user_jid TEXT NOT NULL, taken_at TEXT DEFAULT CURRENT_TIMESTAMP,
    tier INTEGER, foir_bp INTEGER, dcf_paise INTEGER, runway_centimonths INTEGER, debt_paise INTEGER);
`);

const USER_COLS = new Set(["currency","net_income","core_expenses","rent","liquid_reserves",
                           "dependents","earners","family_help","step"]);
updateUser(jid, fields) {
  const keys = Object.keys(fields).filter(k => USER_COLS.has(k));      // whitelist column names
  if (!keys.length) return;
  db.prepare(`UPDATE users SET ${keys.map(k => `${k} = ?`).join(", ")} WHERE jid = ?`)
    .run(...keys.map(k => fields[k]), jid);
},
getDailyExpenses(jid) {
  return db.prepare(`SELECT * FROM expenses
                     WHERE user_jid = ? AND date(created_at, 'localtime') = date('now',
                     ↳ 'localtime')`).all(jid);
},
// Store money as integer paise (Math.round(rupees * 100)); key rows by HMAC(jid, SERVER_SECRET), not the raw
↳ number.
// Add: purgeInactive(days), deleteAllFor(jid) for the DELETE MY DATA command.
```

D.6 Dashboard fixes (`SolvencyDashboard.jsx`, illustrative)

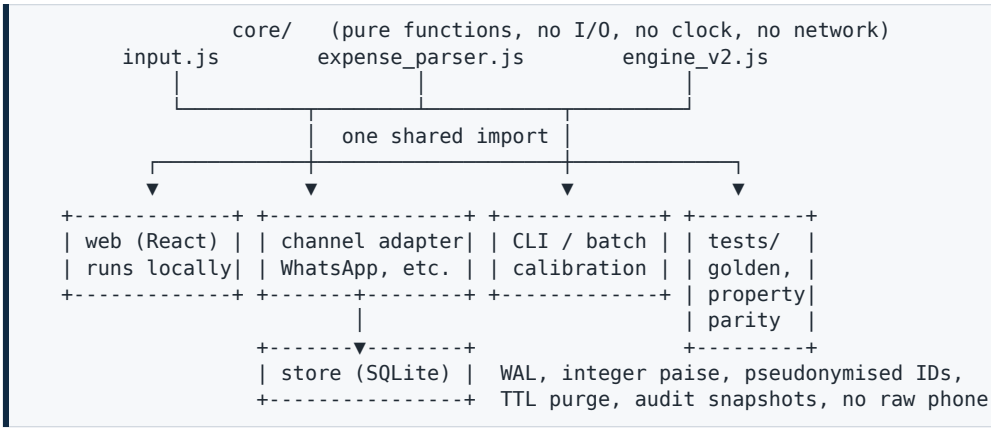
```
// Illustrative patch: not executed.
import { runDiagnosticV2 } from "./core/engine_v2.js";          // one engine for web and bot

const NUMERIC = new Set(["balance", "apr", "minPayment", "creditLimit"]);
const updateDebt = (id, field, value) =>
  setDebts(ds => ds.map(d => d.id !== id ? d : {
    ...d, [field]: NUMERIC.has(field) ? (value === "" ? 0 : Number(value)) : value   // names stay text
  }));

const fmt = n => "₹" + Math.round(n).toLocaleString("en-IN");                      // Indian digit grouping
// Bind labels:  <label htmlFor="income">...</label><input id="income" .../>
// Show tier as text plus colour; show "Plan infeasible: shortfall ₹X/month" when feasible === false.
```

E. Implementation Plan v2

E.1 Target architecture



E.2 Channel decision

Option	Cost	Ban or ToS risk	Privacy	Use when
Baileys, self-chat only (allow-list)	Free	Medium (unofficial client)	Data stays on your machine	Personal prototype for the author
Baileys, dedicated number	Free plus a SIM	High	Users' phone numbers are handled	Short, consented friends-and-family test only
WhatsApp Cloud API (direct or via a provider)	Per Meta pricing; verify free allowance	Low (official)	Needs consent notice and a processing agreement	Any public launch
Telegram bot	Free	Low	Needs user IDs, lighter than phone numbers	Low-friction alternative channel
Offline-first web app or PWA	Free	None	Best: nothing leaves the device	Default zero-PII path

E.3 Roadmap

Phase	Weeks	Deliverables	Exit criteria
0. Stabilise	1 to 2	Fix F-05, 06, 15, 16, 17, 19, 22; extract the shared core; golden fixtures; .gitignore for secrets	Bot and dashboard return identical tiers for 100 random inputs; all fixtures pass
1. Engine v2	3 to 4	Tier policy, true-APR normaliser, payoff v2, strategy selector, plausibility guards, explanation text	Property tests green; distance-to-next-tier shown in reports
2. Privacy and channel	5 to 7	Consent notice, TTL purge, DELETE MY DATA, pseudonymised IDs, allow-list, channel decision, process manager, PII-free logs, encrypted backups	Data inventory (F.1) signed off; legal review done; no raw phone numbers in storage or logs
3. Product loop	8 to 10	Audit history and trend, log-to-audit integration, opt-in nudges, Hinglish, Hindi and Bengali copy, accessibility pass	A user can see tier movement across audits and the 30-day log pre-fills core costs
4. Pilot and calibration	11 to 16	30 to 50 consenting volunteers; threshold review; outreach to non-profit credit counsellors; publish the methodology	Pilot metrics (E.4) reported; thresholds adjusted or confirmed with evidence

E.4 Validation and test plan

Layer	What is tested	Examples
Golden fixtures	Exact outputs for fixed inputs, every band edge	FOIR 39.99, 40, 50, 65; DCF 25, 24.99, 10, 9.99, 0; runway 6, 5.99, 3, 2.99, 1, 0.99; deficit with 5.99 and 6.0 months of reserves
Property tests	Invariants over random inputs	Raising income never worsens the tier; raising any EMI never improves it; scaling every rupee amount by k leaves tier unchanged; payoff interest never negative; budget never below minimums
Parity	Bot and dashboard call the same module	One fixture file consumed by both test suites
Parser fuzz	No crash, no silent mis-scale	Random mixes of commas, symbols, suffixes, Hinglish words; "12abc" must be rejected
Copy snapshots	WhatsApp replies are stable and safe	Snapshot every template; assert no "insolvent" or "prohibited"
Pilot	Does it help?	Audit completion rate; 7 and 30 day return; share taking at least one recommended action; perceived fairness of the verdict; candor vs neutral tone variants

E.5 Sustainability options (no lending, no referrals)

Because the product's integrity rests on having no credit-product revenue, plan funding that preserves that: grants or CSR support from financial-literacy programmes, partnerships with non-profit credit-counselling organisations, a paid white-label licence for employers or NGOs (with no individual-level data sharing), or open-sourcing the core with optional paid hosting. Decide before the pilot so the privacy design is not later bent by revenue needs.

F. Privacy, Regulatory & Safety Posture v2

F.1 Reframe: data-minimised, not "exempt"

Data	Today	v2 control
WhatsApp ID (contains the phone number)	Primary key, plaintext, permanent	HMAC-SHA256 with a server-side secret; raw number never stored; deleted on request
Income, debts, reserves	Plain columns	Integer paise; purge after inactivity (for example 90 days) and on DELETE MY DATA
Expense log	Free-text descriptions	Keep category and amount; free text may contain names or places
auth_info_baileys	Session keys on disk	Restrict file permissions, keep out of git, rotate if exposed
Logs	Silent	Log events, never message bodies

Pseudonymised data generally remains personal data; these controls reduce exposure and breach impact, they do not create an exemption. Show a short notice on first contact stating what is stored, why, for how long, and how to erase it.

F.2 Regulatory notes (verify before launch; this is not legal advice)

- **Data protection (India).** The DPDP Rules, 2025 were notified in November 2025 with a phased commencement: the Data Protection Board provisions took effect immediately, Consent Manager registration follows after about one year, and the main consent, notice, security and breach obligations after roughly 18 months (around May 2027). Plan for notice, purpose-limited consent, security safeguards, breach reporting and erasure from the start.
- **Securities advice.** Keep the tool out of SEBI's investment-adviser perimeter by never naming securities or funds; the v2 Tier 1 text points to a SEBI-registered adviser instead of prescribing investment.
- **Lender legitimacy.** RBI has operated a public directory of digital lending apps deployed by regulated entities since 1 July 2025. Before advising complaints about an app, tell the user how to check whether it is listed; an unlisted app is a different problem from a regulated lender's hardship process.
- **Messaging platform.** Using an unofficial client breaches the platform's terms; the official API has its own business verification and pricing rules, which changed on 1 October 2026 according to provider announcements. Check Meta's current documentation.

F.3 Safety protocol

- **Tone.** State facts and the next step. Use "cash-flow deficit" and "not advised", not "insolvent" or "prohibited". Keep candor about the numbers, not about the person.
- **Crisis escalation.** If a message suggests hopelessness or self-harm, stop the audit flow, respond with care, encourage contacting a trusted person, and share a local helpline (for example Tele-MANAS, 14416, in India; verify the number and its hours before shipping).
- **Harassment.** Tell users to keep screenshots and call logs of abusive recovery contact or threats and to report them through the lender's grievance route, the RBI complaint channels and the national cybercrime reporting channels; verify current contact details.
- **Disclaimers.** Informational only; not legal, tax or investment advice; a qualified professional should review individual circumstances.
- **First contact.** Consent notice, a HELP command, a CANCEL command available at every step, and DELETE MY DATA.

F.4 References and evidence notes

Bibliographic details for the papers are given from memory and should be verified before citing.

- Galai, D. and Sade, O. (2006). The "Ostrich Effect" and the relationship between the liquidity and the yields of financial assets. *Journal of Business*, 79(5). (Working paper circulated from 2003.)
- Karlsson, N., Loewenstein, G. and Seppi, D. (2009). The ostrich effect: Selective attention to information. *Journal of Risk and Uncertainty*, 38(2).
- Gal, D. and McShane, B. (2012). Can small victories help win the war? Evidence from consumer debt management. *Journal of Marketing Research*, 49(4).
- Amar, M., Ariely, D., Ayal, S., Cryder, C. and Rick, S. (2011). Winning the battle but losing the war: The psychology of debt management. *Journal of Marketing Research*, 48(SPL).
- Ministry of Electronics and Information Technology, Digital Personal Data Protection Rules, 2025 (notified November 2025).
- Reserve Bank of India, Digital Lending Directions, 2025 and the public directory of digital lending apps (operational from 1 July 2025).
- Meta, WhatsApp Business Platform pricing documentation; service-message charging change reported as effective 1 October 2026.

G. Recomputed Verification (v2) and Golden Fixtures

G.1 The four archetypes under v2

Reserves are back-solved from the stated runways (A: Rs 10,000; B: Rs 1,81,500; C: Rs 3,00,000; D: nil).

Metric	Profile A	Profile B	Profile C	Profile D
Adjusted FOIR	62.2% (Tier 3)	50.0% (Tier 3)	48.1% (Tier 2)	109.1% (Tier 4)
DCF as % of income	-11.1%; 2.0 months to default (Tier 4)	17.5% (Tier 2)	17.5% (Tier 2)	-63.6%; 0 months (Tier 4)
Emergency runway	0.20 months (Tier 4)	1.10 months (Tier 3)	2.27 months (Tier 3)	0.00 months (Tier 4)
Flow tier	4	3	2	4
Final tier	**Tier 4**	**Tier 3**	**Tier 3 (runway-limited)**	**Tier 4**
Drivers	DCF, Runway	FOIR, Runway	Runway	FOIR, DCF, Runway
Career-risk rule	Applies	Applies	Applies	Applies
Changed vs section 8?	No	No (the shipped code said Tier 2)	**Yes: Tier 2 becomes Tier 3**	No

Profile C is the instructive case: its flow metrics alone say Tier 2, but a 2.27-month buffer on a Rs 1,60,000 income with Rs 1,32,000 of monthly outflow is thin, so v2 raises it one level and says why. If the author prefers to keep runway advisory, set `runwayMaxPenalty` to 0 in `POLICY`; the point is that the choice is now explicit and tested rather than accidental.

G.2 Boundary fixtures to commit

Metric	Value to test	Expected tier
FOIR	39.99 / 40 / 50 / 65	1 / 2 / 3 / 4
DCF (% of income)	25 / 24.99 / 10 / 9.99 / 0	1 / 2 / 2 / 3 / 3
Deficit (reserves / monthly shortfall)	5.99 / 6.0	4 / 3
Runway (months)	6 / 5.99 / 3 / 2.99 / 1 / 0.99	1 / 2 / 2 / 3 / 3 / 4
Overrides	Minimum below monthly interest	At least Tier 3
Overrides	Dependents / (earners + dependents) at 0.5 or more, runway under 3	Tier plus one, capped at 4

G.3 What to do first

1. Decide the channel (E.2) before writing more bot code; it determines the privacy design.
2. Extract the shared core and commit the fixtures in G.2 so every later change is checked.
3. Fix the four silent-error bugs (F-05, F-06, F-16, F-17) because they change what users are told without any visible failure.
4. Rewrite the privacy claim and add the first-contact notice before any person other than the author uses the bot.